

MLSYS · IM

First-Principles Infrastructure Modeling for Machine Learning Systems

Vijay Janapa Reddi

Harvard University

mlsysbook.ai/mlsysim

Abstract

As machine learning shifts from laboratory curiosity to critical infrastructure, the systems that sustain it span an extraordinary range, from sub-milliwatt microcontrollers to multi-gigawatt datacenter fleets. Reasoning across this range is hard: empirical profiling requires the target hardware in hand, while cycle-level simulation can take minutes to hours per configuration, leaving no tool for rapid, full-stack architectural reasoning. We present **MLSYS · IM** (Machine Learning Systems Infrastructure Modeling), a first-principles analytical framework that formalizes the “physics of systems” into a dimensionally-strict Python engine. MLSYS · IM is built on a *demand–supply* abstraction that decouples computational demand from silicon supply and environmental context, and it carries units on hardware, workload, and network quantities at runtime, so mixing a compute rate with a memory bandwidth raises an error instead of silently producing a wrong number. Registry values carry typed provenance records that an audit tool checks. On this engine we codify a taxonomy of 22 “Systems Walls” resolved by 28 composable models, solvers, and optimizers, enabling sub-second design-space exploration that identifies binding constraints and derives minimum hardware specifications across the ML systems lifecycle.

1 Introduction

Progress in machine learning has come from methods that exploit ever more computation (Sutton, 2019), and that computation now runs as infrastructure. Pre-training Llama 3 405B used up to 16,384 H100 GPUs (Llama Team, AI Meta, 2024), spread across datacenter fabrics where memory ceilings, network bandwidth, power delivery, and regional carbon intensity interact in non-obvious ways. Model scale has grown just as quickly, with sparse models passing a trillion parameters (Fedus et al., 2022).

Yet the hardware required to develop intuition for these systems is prohibitively scarce: a student cannot requisition a 100,000-GPU cluster to explore how topology affects AllReduce latency, and a researcher cannot easily sweep parallelism strategies across hardware generations. This creates a growing *reasoning gap*. The systems are getting more complex, but the tools to think about them have not kept pace.

Consider a concrete example. A team deploying LLaMA-3 70B for interactive serving must answer: *How many H100 GPUs are needed to meet a 50 ms time-to-first-token SLA at 95th-percentile latency?* The answer depends on at least six interacting constraints, each a “wall” (a hard bound imposed by physics, economics, or algorithmic scaling; see Table 1). We say a wall *binds* when it is the tightest constraint, the single bottleneck that limits end-to-end performance. The *binding constraint* is the wall whose relaxation would yield the largest throughput improvement; all other walls are slack.¹ For the LLaMA-3 serving example, the model’s 70 billion parameters require ~141 GB in FP16, exceeding a single GPU’s 80 GB HBM capacity (Wall 2: Memory). Tensor-parallel sharding across two GPUs introduces NVLink synchronization overhead (Wall 14: Communication). Continuous batching with PagedAttention determines KV-cache memory utilization (Wall 5: Batching). The decode phase is memory-bandwidth-bound at 3.35 TB/s per device (Wall 4: Serving). Tail latency under load follows Erlang-C queueing dynamics (Wall 7: Tail Latency). And the fleet’s total cost of ownership constrains what is economically viable (Wall 17: Capital). A similar multi-wall analysis applies to training: determining the optimal parallelism strategy for a 512-GPU run involves compute throughput, memory capacity, communication overhead, scaling laws, reliability, and economics simultaneously. No single equation suffices.

¹This usage mirrors linear programming: a constraint *binds* (is *active*) when the optimal point lies on it. Equivalently, in Roofline analysis, the binding ceiling is the one that determines achievable performance.

Existing tools are constrained along three axes (fidelity, speed, and scope), and no tool occupies the region where all three are adequate (Table 2). Profilers require physical silicon; system simulators such as ASTRA-sim 2.0 (Won et al., 2023) reach cycle-level network fidelity at a cost of about twenty minutes to simulate a single collective; analytical tools like Calculon (Isaev et al., 2023) achieve speed but focus narrowly on LLM training, ignoring data pipelines, reliability, sustainability, and inference. Computer architecture education faced an analogous trade-off, and simplified instruction-set models such as MIPS became standard teaching vehicles because they are simple enough to reason about yet faithful enough to build correct intuition. MLSYS·IM makes the same choice for ML systems. Beyond coverage gaps, the tools surveyed here do not check units. In ML systems, confusing GB with GiB, FLOP/s with FLOPs, or bandwidth with throughput can silently invalidate a capacity-planning spreadsheet.

MLSYS·IM (Machine Learning Systems Infrastructure Modeling) is an open-source, pure-Python, first-principles analytical modeling framework for ML systems, designed for education and early design-space reasoning before empirical benchmarking. It formalizes back-of-the-envelope ML systems reasoning into a dimensionally strict, composable engine. The framework codifies 22 “Systems Walls” into 28 composable resolvers organized across six domains: Node, Data, Algorithm, Fleet, Operations, and Analysis. It separates computational *demand* from silicon *supply* and environmental *context* through a 5-layer demand–supply architecture, enforces unit correctness at runtime via the `pint` library, and produces full-stack analysis in milliseconds on any laptop.

Each of the resolvers computes structural physics from first-principles equations and hardware datasheet constants, then bridges the gap to measured performance through a single, explicit efficiency coefficient η that absorbs second-order effects (kernel launch overhead, straggler variance, framework dispatch costs). This is the same abstraction that makes the Roofline model useful: replacing peak bandwidth with *achievable* bandwidth sacrifices cycle-level precision for the ability to reason correctly about which constraint binds and why (Williams et al., 2009).

MLSYS·IM serves three communities. *Students* build quantitative intuition through hands-on tutorials where changing a single parameter reveals which wall binds and why. *Instructors* run live classroom demonstrations that produce concrete numbers in real time, replacing hand-waving with Roofline diagrams. *Researchers* perform rapid what-if analysis by comparing parallelism strategies, evaluating procurement decisions, or pro-

jecting carbon footprints before committing resources. Designed as the analytical companion to the *Machine Learning Systems* textbook (Reddi et al., 2025a) (see Appendices A and B for pedagogical integration examples and registry snapshots), MLSYS·IM is fully open-source (Apache-2.0 license) with deterministic, hardware-free execution, and a committed script regenerates every number and data figure in this paper from the released package. This matters for educational equity. A student at a community college with a Chromebook can run the same quantitative exercises and verify the same binding constraints as one with access to a university GPU cluster. We validate against published empirical anchors spanning five taxonomy domains, matching values within 7% error while sweeping over 1,000 configurations in under one second.

This paper makes the following contributions:

- C1. A Taxonomy of 22 Systems Walls** (Table 1), each grounded in a published equation and mapped to a resolver. The taxonomy provides a structured, deliberately bounded vocabulary for reasoning about the constraints that bind ML system performance (Section 4).
- C2. Demand–Supply Separation with Dimensional Strictness.** A 5-layer architecture formally decouples computational demand from silicon supply through a single workload-lowering step. Hardware, workload, and network quantities carry units at runtime through `pint`, turning dimensional analysis on those paths from a manual discipline into a machine-checked invariant (Section 3).
- C3. Composable Resolver Algebra.** 28 resolvers (23 models, 2 solvers, and 3 optimizers) compose through chaining: each is a pure function $f(\text{config}) \rightarrow \text{metrics}$, producing a three-level evaluation (Feasibility, Performance, Macro) that identifies binding constraints. The algebra includes an inverse-Roofline *synthesis* solver that derives minimum hardware specifications from SLA requirements (Sections 5 and 7).
- C4. Accessible Full-Stack Reasoning without Hardware.** MLSYS·IM runs on any laptop without GPUs, clusters, or cloud credits. It powers deterministic labs and browser-friendly examples, providing an accessible systems engineering environment for students and instructors (Section 7).

The paper builds the framework in layers. We first survey the modeling landscape and identify the void (Section 2). Section 3 presents the architecture, including four design principles, a 5-layer input stack, the

Table 1: **The 22 ML Systems Walls.** Each wall represents a physical or logical constraint resolved by a dedicated resolver (model or solver). Walls 1–2 (Compute and Memory) share the `SingleNodeModel`; four companion models deepen individual walls (`TrainingMemoryModel` for Wall 2, `ServingCapacityModel` for Wall 4, `MoERoutingModel` for Wall 14, and `NetworkRooflineModel`, which lifts the Roofline to fleet scale). Together with 3 optimizers (parallelism, batching, placement), the framework provides 28 resolvers across 22 walls. Domains progress from local node resources through data movement and algorithmic scaling to fleet coordination, operations, and cross-cutting analysis. Each wall is formalized in Section 4.

#	Wall	Resolver	Bounded	Core Equation	Ref.
<i>Node (Single-Accelerator Resources)</i>					
1	Compute	SingleNode	Peak FLOP/s	$T = \text{OPs} / (\text{Peak} \times \eta)$	Williams et al. 2009
2	Memory	SingleNode	HBM BW + cap.	$T = W / BW_{\text{HBM}}$	Williams et al. 2009
3	Software	Efficiency	Achieved MFU	$\eta = \text{FLOPS}_{\text{ach}} / \text{Peak}$	Chowdhery et al. 2023
4	Serving	Serving	Prefill vs. dec.	$T_{\text{pf}} = 2PS / (F\eta); T_{\text{dec}} = W / BW$	Pope et al. 2023
5	Batching	Cont. Batch	KV allocation	$N_{\text{st}} = \lfloor M_{\text{KV}} / (kS_{\text{max}}) \rfloor; N_{\text{pg}} = \lfloor M_{\text{KV}} / (kp \mathbb{E}[S/p]) \rfloor$	Kwon et al. 2023
6	Streaming	WeightStream	Injection BW	$T = \max(W_{\ell} / BW, 2P_{\ell}B / F\eta)$	Lie 2022
7	Tail Latency	TailLatency	P99 queueing	Erlang-C $M/M/c$	Harchol-Balter 2013
<i>Data (Movement & Pipelines)</i>					
8	Ingestion	Data	Storage I/O	$\rho = BW_{\text{demand}} / BW_{\text{supply}}$	Mohan et al. 2021
9	Transform.	Transform.	CPU preproc.	$T = B \cdot S_{\text{sample}} / C_{\text{cpu}}$	Murray et al. 2021
10	Locality	Topology	Bisection BW	$BW_{\text{eff}} = BW_{\text{link}} \cdot \beta / \text{osub}$	Leiserson 1985
<i>Algorithm (Scaling & Compression)</i>					
11	Complexity	Scaling	Scaling laws	$C = 6PD; P^* = \sqrt{C/120}$	Hoffmann et al. 2022
12	Reasoning	Inf. Scaling	Inf.-time comp.	$T = K \times T_{\text{step}}$	Snell et al. 2025
13	Fidelity	Compression	Acc.-efficiency	$r = b_{\text{base}} / b; r = 1 / (1-s)$	Han et al. 2016
<i>Fleet (Multi-Node Coordination)</i>					
14	Communic.	Distributed	AllReduce	$T = 2 \frac{N-1}{N} \frac{M}{B_{\text{link}}} + 2(N-1)\alpha$	Patarasuk and Yuan 2009
15	Fragility	Reliability	Cluster MTBF	$\text{MTBF}_{\text{cl}} = \text{MTBF}_{\text{node}} / N$	Daly 2006
16	Multi-tenant	Orchestration	Queue wait	$T_{\text{wait}} = \rho / [2\mu(1-\rho)]$	Harchol-Balter 2013
<i>Operations (Economics, Sustainability & Safety)</i>					
17	Capital	Economics	TCO	$\text{TCO} = \text{CapEx} + \text{OpEx}$	Barroso et al. 2018
18	Sustain.	Sustainability	Carbon + water	$\text{CO}_2 = E \times \text{PUE} \times \text{CI}$	Patterson et al. 2021
19	Checkpoint	Checkpoint	I/O burst penalty	$\text{penalty} = T_{\text{write}} / T_{\text{interval}}$	Eisenman et al. 2022
20	Safety	Resp. Eng.	DP-SGD overhead	$\sigma \propto 1/\epsilon$	Abadi et al. 2016
<i>Analysis (Cross-Cutting Diagnostics)</i>					
21	Sensitivity	Sensitivity	Binding constr.	$\partial T / \partial x_i$	Williams et al. 2009
22	Synthesis	Synthesis	Inverse spec	$BW_{\text{req}} = W / T_{\text{target}}$	Williams et al. 2009
<i>Optimizers (Design-Space Search, no dedicated wall)</i>					
	Parallelism	Parallelism	Max MFU	$\max_c \eta(c) \text{ over } \text{TP} \times \text{PP} \times \text{DP}$	Shoeybi et al. 2019
	Batching	Batching	SLA-aware B^*	$\max B \text{ s.t. } P_{99} \leq T_{\text{SLA}}$	—
	Placement	Placement	Cost-perf.	$\min \text{cost s.t. } T \leq T_{\text{SLA}}$	—

dimensionally strict type system, and the extensibility model. Section 4 formalizes the 22-wall taxonomy, and Section 5 defines the 3-tier resolver algebra (Models, Solvers, Optimizers) that composes over it. We validate against published benchmarks spanning five domains (Section 6), demonstrate the framework through student, instructor, and researcher use cases (Section 7), surface

common misconceptions the framework is designed to expose (Section 8), discuss limitations and future work (Section 9), and conclude (Section 10).

2 Related Work

Tools for modeling and evaluating ML systems span a wide spectrum of fidelity, scope, and intended audience. We organize prior work into four categories and position MLSYS·IM relative to each: (1) cycle-level simulators that model hardware at the microarchitectural level, (2) accelerator design tools that evaluate individual chip architectures, (3) analytical and co-design tools that trade fidelity for speed, and (4) pedagogical simulators that prioritize conceptual clarity. Table 2 provides a quantitative summary across these categories.

2.1 Cycle-Level Simulators

Cycle-accurate simulators provide the highest fidelity by modeling hardware behavior at the microarchitectural level. gem5 (Binkert et al., 2011) is the canonical general-purpose architecture simulator, capable of modeling processors down to individual pipeline stages. While invaluable for processor design, gem5 lacks ML-specific abstractions (it has no notion of a transformer layer, a training step, or a parallelism strategy), and full-system simulation at that level of detail is slow enough to make broad design-space sweeps impractical.

ASTRA-sim 2.0 (Won et al., 2023) addresses the ML gap with a hierarchical system simulator purpose-built for distributed training. It models collective communication patterns such as AllReduce across realistic network topologies through pluggable network backends. Its cycle-level Garnet backend took 21.42 minutes to simulate a 1 MB AllReduce on 64 NPUs, while its analytical backend ran the same case in 1.70 seconds. SimAI (Wang et al., 2025) extends this approach with a full-stack training simulator that integrates NS3-based network modeling with kernel computation traces, reporting an average of 98.1% alignment with real-world results on clusters of up to 1,024 GPUs. Both pay for fidelity in time: detailed communication simulation at cluster scale takes minutes or more per configuration, which makes broad iterative design-space exploration impractical. Their scope is also limited to communication and compute; neither models economics, sustainability, data pipelines, or reliability.

MLSYS·IM occupies a different point on the fidelity-speed spectrum. Where ASTRA-sim answers “how many microseconds does this AllReduce take on this exact topology,” MLSYS·IM answers “which of 22 possible bottlenecks binds this system, and how does changing hardware shift the binding constraint?” The two classes are complementary: MLSYS·IM narrows the design space, and high-fidelity simulators validate specific points within it.

2.2 Accelerator Design Tools

A second class of tools targets the design and evaluation of individual accelerator architectures. Timeloop (Parashar et al., 2019) provides a systematic methodology for evaluating DNN accelerator dataflows, modeling how data tiles map onto spatial architectures and estimating latency and energy for each mapping. Accelergy (Wu et al., 2019), its companion framework, supplies the energy estimation primitives that Timeloop consumes. Together, they form a powerful toolkit for accelerator architects exploring the design space of novel silicon. LLMCompass (Zhang et al., 2024) brings this approach to LLM inference, combining an automated mapper with an area-based cost model to explore compute, memory bandwidth, and buffer configurations, achieving 4.1% error across prefill and decoding on a 4-GPU A100 node, with a GPT-3 evaluation taking 16 minutes.

These tools operate at the operator and tile level, modeling how a single convolution or matrix multiplication executes on a specific microarchitecture. They do not reason about fleet-level concerns such as how the data pipeline feeds a cluster or what the total cost of ownership looks like at fleet scale. MLSYS·IM operates one abstraction level higher. It consumes the *outputs* of accelerator-level analysis (peak FLOP/s, memory bandwidth, TDP) as inputs to its hardware registry and reasons about how those specifications interact with workload demands, network topologies, and infrastructure constraints.

2.3 Analytical and Co-Design Tools

Closest in spirit to MLSYS·IM are analytical tools that sacrifice microarchitectural detail for speed. Calculon (Isaev et al., 2023) is an analytical co-design tool for large language model training. It models training time as a function of hardware specifications, parallelism strategies, and model architecture, achieving execution speeds comparable to MLSYS·IM. However, Calculon’s scope is narrow by design, targeting transformer-based LLM training exclusively, with no support for CNNs, mixture-of-experts architectures, or inference workloads. It does not model data pipelines, reliability, sustainability, economics, or safety considerations, and it lacks dimensional enforcement. Lumos (Liang et al., 2025) takes a trace-driven approach, using profiled kernel traces to predict LLM training performance at scale with 3.3% average error on up to 512 H100 GPUs. While Lumos achieves higher single-point accuracy than MLSYS·IM, it requires empirical traces from the target hardware, limiting its use in what-if exploration across hypothetical configurations. The DeepSeek-V3 systems paper (DeepSeek-AI, 2025) exemplifies the kind

Table 2: **Comparison of ML systems modeling tools.** *Scope*: which system aspects a tool models (e.g., “compute & comm.” means compute throughput and collective communication only, while “full-stack” spans all six taxonomy domains). *Speed*: typical wall-clock time for a single evaluation. *Walls*: our assessment, from each tool’s published scope, of how many of the 22 Systems Walls (Table 1) it models. *Phase*: whether training, inference, or both are modeled. *Dist.*: whether multi-node (multi-accelerator) distributed analysis is supported.

Tool	Approach	Scope	Speed	Walls	Phase	Dist.
<i>Cycle-level</i>						
gem5	Cycle-accurate	CPU/GPU microarchitecture	Hours	1–2	Both	–
ASTRA-sim 2.0	System sim.	Network collectives, topology	Sec.–min.	1–2	Train	✓
SimAI	Trace-driven	Full-stack distributed training	Minutes	2–3	Train	✓
<i>Accelerator design</i>						
Timeloop + Accelergy	Analytical	Accelerator dataflow & energy	Minutes	1–2	Infer	–
LLMCompass	Analytical	LLM inference HW design space	Minutes	2–3	Infer	◦
<i>Analytical & co-design</i>						
Paleo	Analytical	DNN training compute & comm.	Seconds	2	Train	✓
Calculon	Analytical	LLM training performance	Seconds	2–3	Train	✓
Lumos	Trace-driven	LLM training perf. modeling	Seconds	2–3	Train	✓
Vidur	Empirical	LLM inference scheduling	Seconds	3–4	Infer	–
GenZ	Analytical	LLM inference platform design	Seconds	2–3	Infer	–
LLM-Viewer	Analytical	LLM inference memory/latency	Seconds	1–2	Infer	–
<i>Sustainability</i>						
LLMCarbon	Analytical	LLM carbon footprint (op. + embodied)	Seconds	1	Both	–
CodeCarbon	Empirical	Runtime energy & carbon tracking	Seconds	1	Both	–
MLSYS · IM	Analytical	Full-stack: compute, memory, network, data, scaling, reliability, econ., sustainability, safety	Sub-sec.	22	Both	✓

of hardware-aware co-design analysis that MLSYS · IM targets: it demonstrates how FP8 mixed-precision training, MoE sparsity, and multi-plane network topology interact to achieve frontier-model training at a fraction of conventional cost, a multi-wall optimization that spans Walls 1, 3, 13, 14, and 17 in our taxonomy.

Paleo (Qi et al., 2017) pioneered the analytical approach, decomposing DNN training time into computation and communication components across data- and model-parallel configurations. FlexFlow (Jia et al., 2019) optimizes parallelism strategies through simulation-guided search, and Habitat (Yu et al., 2021) provides cross-hardware extrapolation of training performance using execution-time scaling curves. While these tools advance specific aspects of analytical modeling, they predate or do not address the full scope of modern concerns (inference serving, fleet economics, sustainability). Vidur (Agrawal et al., 2024b) extends analytical modeling to LLM *inference*, using operator-level profiling to build a fine-grained runtime estimator validated at less than 5% error across multiple LLMs and scheduling policies. GenZ (Bambhaniya et al., 2024) provides an analytical framework for LLM inference platform design that models multi-dimensional network topologies

and serving optimizations. DistServe (Zhong et al., 2024) and Sarathi-Serve (Agrawal et al., 2024a) advance LLM serving through prefill-decode disaggregation and chunked-prefill scheduling respectively, demonstrating that the two-phase inference model (Equation (7)) requires increasingly sophisticated scheduling to achieve high goodput. The inference tools in this group (Vidur, GenZ, DistServe, and Sarathi-Serve) do not model training, and none of the tools in this paragraph models data pipelines, economics, or sustainability.

LLM-Viewer (Yuan et al., 2024) and llm-analysis (Li, 2023) provide lightweight memory and latency estimation for transformer training and inference. These tools are useful for single-model profiling but do not extend to fleet-level reasoning, multi-tenant scheduling, or cross-domain constraint analysis.

A parallel line of work targets sustainability and fleet efficiency. LLMCarbon (Faiz et al., 2024) projects end-to-end carbon footprints (operational and embodied) for dense and MoE LLMs, reporting differences of at most 8.2% from the published footprints it was checked against. CodeCarbon (Courty et al., 2024) tracks the energy use and carbon emissions of code as it runs. Wongpanich et al. (2025) introduce *ML Productivity Goodput*

(MPG) as a fleet-level efficiency metric for warehouse-scale TPU clusters, demonstrating that traditional utilization metrics are insufficient for characterizing ML fleet performance across model, data, framework, compiler, and scheduling layers. These tools each address one or two domains and do not model the full cross-stack interactions that determine *why* a workload consumes so much energy.

MLSYS·IM generalizes the analytical approach to the full ML systems stack. Where each tool above models one or two domains, MLSYS·IM composes resolvers spanning all six (Table 1): compute, memory, serving, data pipelines, scaling laws, fleet coordination, economics, sustainability, and responsible engineering. Runtime dimensional strictness on hardware, workload, and network quantities is the invariant of Section 3.1.2.

2.4 Pedagogical Precedents

MLSYS·IM draws direct inspiration from the tradition of pedagogical simulators in systems education. SPIM, James Larus’s MIPS simulator used alongside Patterson and Hennessy’s textbook (Patterson and Hennessy, 2014), taught generations of students computer architecture not by replicating a production processor, but by running programs on a simplified instruction-set model that made architectural concepts tangible through rapid experimentation. Similarly, xv6 (Cox et al., 2011) and MINIX (Tanenbaum and Woodhull, 2006) teach operating systems by stripping away production complexity to reveal core abstractions.

MLSYS·IM follows this pedagogical philosophy for the ML systems domain. Production ML infrastructure (spanning millions of lines of code across frameworks, compilers, schedulers, and orchestrators) is too complex for students to reason about directly. MLSYS·IM provides a controlled environment in which students can sweep hardware configurations, vary parallelism strategies, and observe how binding constraints shift, all in rapid iteration cycles. Its integration with the companion textbook (Reddi et al., 2025a) supports structured laboratory exercises, which the research-oriented tools surveyed in Section 2 do not target.

3 Architecture

Figure 1 presents the end-to-end framework. This section unpacks the design in five parts. Section 3.1 introduces the four principles (analytical speed, dimensional strictness, taxonomic coverage, demand–supply separation) that constrain every subsequent decision. Section 3.2 translates those principles into a five-layer input stack of workloads, hardware, infrastructure, topology, and resolvers. Section 3.3 describes the dimensionally

strict type system that enforces correctness at every API boundary, Section 3.4 explains how new workloads, hardware, and resolvers compose with the existing pipeline without modification, and Section 3.5 maps the public import surface and the presentation layer that sits outside the analytical engine.

3.1 Design Philosophy

The central question MLSYS·IM addresses is: *Where does the complexity of an ML system come from?* We argue that complexity arises from the non-linear interaction of constraints across six distinct domains (node resources, data movement, algorithms, fleet coordination, operations, and cross-cutting analysis) and that an effective modeling tool must formalize *all six* simultaneously. Four design principles govern MLSYS·IM’s approach.

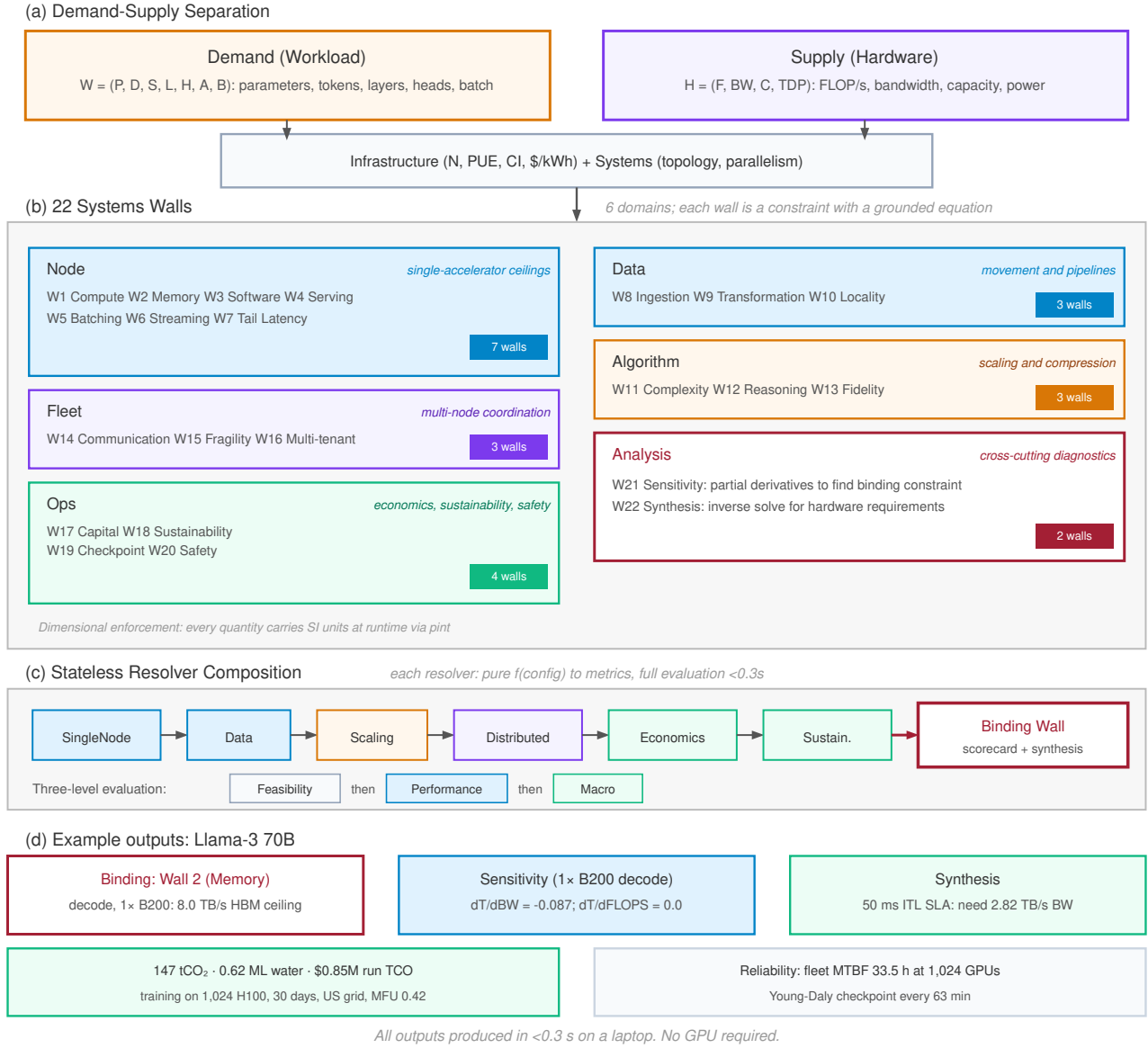
3.1.1 Analytical Speed over Cycle Accuracy

Analytical models that execute in sub-second time enable iterative design-space exploration that cycle-accurate simulators cannot support at comparable speed. By absorbing microarchitectural detail (cache hit rates, kernel scheduling) into a single efficiency parameter η , MLSYS·IM achieves sub-second execution per solve. This enables sweeps over thousands of hardware–model–topology combinations in seconds, the kind of rapid “what-if” exploration that Hennessy et al. (2024) identify as necessary for quantitative architectural reasoning.

To maintain rigor despite analytical simplification, MLSYS·IM enforces a **“No Magic Numbers” invariant**. Registry values carry a typed `Provenance` record whose *kind* (datasheet, literature, industry report, convention, estimate, derived, illustrative, or heuristic) determines what evidence it must supply: evidence-backed kinds require a source URL and a verification date, while estimates and derived values require explanatory notes, and all records require a dated verification. The H100’s FP16 rate is not a bare 989 floating-point literal; it is $989 * \text{TFLOPs} / \text{second}$, half of the 1,979 teraFLOPS FP16 Tensor Core figure that NVIDIA publishes *with sparsity* (NVIDIA Corporation, 2023). Shared sources live once in a central catalog of 175 provenance records (`core.provenance_catalog`) that registry entries reference by key, and an audit tool (`python -m mlsysim.tools.audit_provenance`) walks every registry and fails on any value with missing or weak provenance. This discipline ensures that analytical speed does not come at the cost of reproducibility.

3.1.2 Dimensional Strictness as an Invariant

Dimensional consistency is a pervasive challenge in systems modeling. Mixing gigabits with gigabytes, or omitting the refractive index of fiber in latency calculations, are canonical failure modes that silently corrupt



```
5 rate + bw # → raises DimensionalityError
```

Listing 1: **Dimensional Strictness.** Prevents silent unit errors at the API level.

This design eliminates the single most common class of bugs in back-of-the-envelope systems analysis, namely silent unit conversion errors.² Hardware and model specifications live in typed registries (e.g., `Hardware.Cloud.H100.memory.bandwidth = 3.35 * TB/s`); the unit vocabulary lives in `core.units`, and physical constants and formulas live in `mlsysim.physics`, preventing unit mismatches at the API level. The physics formulas additionally validate the *dimensionality* of their arguments, raising `DimensionalityError` when, for example, a byte count is passed where a bandwidth is expected.

3.1.3 Taxonomic Coverage

We aim for broad coverage of the bottlenecks that have stable analytical formulations rather than for completeness, and Section 9 lists the constraints we deliberately exclude. `MLSYS·IM` codifies 22 such bottlenecks, which we call *Systems Walls*, organized into six domains: Node (single-accelerator resources), Data (movement and pipelines), Algorithm (scaling and compression), Fleet (multi-node coordination), Operations (economics, sustainability, and safety), and Analysis (cross-cutting diagnostics). Each wall maps to a resolver with a formal equation grounded in the systems literature. Section 4 presents the full taxonomy with formal definitions, and Section 5 describes how solvers compose.

3.1.4 Demand–Supply Separation

`MLSYS·IM` enforces a strict separation between *what* a model computes and *where* it runs. A `TransformerWorkload` describes computational demand (parameters, layers, FLOPs, arithmetic intensity) without reference to any specific accelerator. A `HardwareNode` describes physical supply (peak throughput, memory bandwidth, TDP) without reference to any specific model. This decoupling, inspired by the compiler IR philosophy of separating representation levels (Hennessy et al., 2024), enables hardware–software co-design. The same GPT-3 workload can be evaluated against an H100, a TPU v5p, or a hypothetical future accelerator in a single parametric sweep, with all dimensional conversions handled automatically.

²The most infamous unit-conversion failure is the Mars Climate Orbiter, lost in 1999 because ground software produced thrust data in pound-force seconds while the spacecraft expected newton seconds (Stephenson et al., 1999).

Table 3: **Supported Workload Types.** Each workload lowers to a `ComputationGraph` with total FLOPs, weight bytes, and arithmetic intensity.

Workload	Key Parameters	Scaling
Transformer	P, L, H, D , seq. length	$2P$ FLOPs/token
CNN	P , inference FLOPs	Fixed per image
Sparse (MoE)	Total vs. active P , experts	Active P for FLOPs
SSM (Mamba)	P , state dim, D	$O(1)$ state cache
Diffusion	P , denoising steps T	$T \times$ FLOPs/step

3.2 The Layered Input Stack

`MLSYS·IM` implements these design principles through a five-layer architecture (Figure 2). Layers A–D are *input layers* that describe demand, supply, context, and topology respectively. Layers A–C are independent of one another, and Layer D composes Layer B accelerators and, optionally, a Layer C grid or datacenter into a fleet. The single lowering step occurs in Layer A, where a workload’s `lower()` method produces a hardware-agnostic `ComputationGraph` intermediate representation. Layer E (Resolvers) then consumes any combination of layers A–D as needed; a single-node analysis requires only A+B, while a fleet-wide carbon estimate draws on A+B+C+D.

Layer A: Workloads (Demand). A workload is a hardware-agnostic description of computational demand. `MLSYS·IM` provides five concrete workload types (Table 3), each exposing a `lower()` method that produces a `ComputationGraph` (an intermediate representation containing total operations, weight bytes, and arithmetic intensity in `flop/byte`). This IR is the contract between demand and supply, capturing *what* must be computed without prescribing *how*.

Layer B: Hardware (Supply). A `HardwareNode` composes four subsystems: `ComputeCore` (peak FLOP/s at the base FP16 precision plus a precision-keyed dictionary, e.g., TF32, FP8, INT8, FP32), `MemoryHierarchy` (capacity and bandwidth), optional `StorageHierarchy`, and optional `IOInterconnect`. Each node also carries TDP, unit cost, and a kernel dispatch tax. The `ridge_point()` method computes the Roofline inflection $R = F_{\text{peak}}/BW_{\text{mem}}$ in `flop/byte` (Williams et al., 2009), enabling immediate classification of any lowered workload as compute-bound or memory-bound.

Layer C: Infrastructure (Context). `GridProfile` objects encode regional environmental parameters: carbon intensity (gCO_2/kWh), Power Usage Effectiveness (PUE), and Water Usage Effectiveness (WUE). A `Datacenter` composes a grid profile with an optional

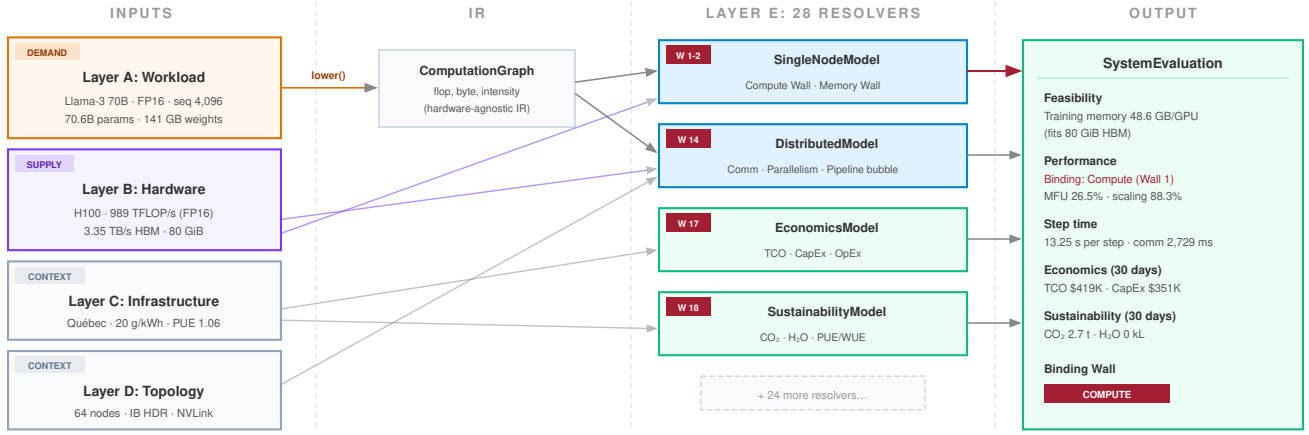


Figure 2: The MLSys · IM 5-Layer Architecture. Layers A–D provide typed inputs: workload demand, hardware supply, infrastructure context, and network topology. The single lowering step occurs in Layer A, where `lower()` produces a hardware-agnostic Computation Graph (an intermediate representation of total FLOPs, weight bytes, and arithmetic intensity). Layer E’s 28 stateless resolvers consume these inputs and produce a three-level SystemEvaluation scorecard. The red arrow marks the binding constraint identified by the solver chain.

facility PUE override; rack power density is described separately by `RackProfile`. This layer converts raw energy consumption into carbon footprint and water usage, following the methodology of [Patterson et al. \(2021\)](#).

Layer D: Systems (Topology). A `Fleet` composes `Nodes` (each a physical compute server containing one or more accelerators on PCIe or NVLink) with a `NetworkFabric` (topology, inter-node bandwidth, latency, oversubscription ratio). This layer enables distributed analysis. The `DistributedModel` decomposes workloads using 4D parallelism (data-parallel DP $\times$ tensor-parallel TP $\times$ pipeline-parallel PP $\times$ expert-parallel EP) and calculates hierarchical AllReduce costs, pipeline bubble fractions, and scaling efficiency.

Layer E: Resolvers (Analysis). Twenty-eight stateless resolvers (23 models, 2 solvers, and 3 optimizers) consume demand, supply, context, and topology to produce dimensioned performance metrics. The solver formalism (stateless composition, chaining semantics, and three-level evaluation) is detailed in Section 5.

The five layers are useful only if their inputs are reproducible. MLSys · IM therefore organizes vetted specifications into eight curated registries collectively called the *MLSys Zoo*: **Hardware** (accelerators and boards, plus technology classes under `Hardware.Tech`), **Models** (workload architectures across six family files: language, vision, generative vision, recommendation, state-space, and TinyML), **Datasets** (benchmark catalog entries), **Platforms** (deployment envelopes), **Infrastructure** (grids, datacenters, racks, cooling, pricing, and capacity),

Systems (nodes, racks, fabrics, clusters, pods, and storage paths), **Ops** (operational policies and overhead profiles such as training-run goodput budgets and monitoring thresholds), and executable **Scenarios** that compose existing model, hardware, infrastructure, and system entries with local constraints such as latency, power, run length, or region without redefining the underlying facts. Two extension registries, **Agents** (agentic workload profiles) and **Embodied** (robot platforms), sit outside the wall taxonomy; Section 9 explains why. Two auxiliary surfaces complete the picture: non-executable real-world anchors, such as Waymo data-rate ranges or mobile power envelopes, live in `ReferenceStats.*`, and cited field figures (MFU bands, scaling laws) live in `Literature.*`. Solver fallbacks live in `mlsysim.engine.calibration`; architecture formulas (Roofline, AllReduce, Young-Daly) live in `mlsysim.physics`; the machine-readable wall taxonomy of Table 1 ships as `mlsysim.engine.walls`; and domain-reviewed empirical anchors in `mlsysim.engine.empirical` bind registry entries to published benchmark envelopes for regression testing.

Registry entries are authored as YAML data files and validated against `pydantic` schemas at import time, so stating a fact is a data edit while computing or composing remains Python. The loader rejects duplicate keys outright and resolves two reference forms: `@tech:` pointers that let an instance inherit a technology-class fact, and `@prov:` pointers into the shared provenance catalog (Section 3.1.1). The `Hardware.Tech` sub-registry encodes the load-bearing distinction between

instances and *technology classes*: per-part facts that vary between products (capacity, bandwidth, TDP, price) live on instances such as `Hardware.Cloud.H100`, while facts that are properties of a technology generation (access latency, energy per operation, interconnect latency) live once on tech-class entries. For example, `Hardware.Cloud.H100` takes its NVLink latency from `Hardware.Tech.Interconnect.NVLink` through an `@tech:` reference, and instance lookups fall back to the tech class when a value is unset. Each registry entry is a fully typed object; for example, `Hardware.Cloud.H100` returns a `HardwareNode` with all physical quantities dimensioned via `pint`. Figure 3 illustrates how these reusable facts become a concrete infrastructure model: a scenario binds a workload, accelerator fleet, regional grid, and operational policy to a local question, then passes the resulting typed objects to the resolver chain.

Once a scenario has assembled typed inputs, the `physics` package decomposes the framework’s analytical equations into eleven domain-aligned modules (Table 4), each owning one conceptual domain of the systems stack, plus two supporting modules: `physics.constants` holds universal physical constants (e.g., the speed of light in fiber), and `physics.quantities` provides generic dimensioned helpers (transfer time, compute time, energy from power, carbon from energy). A shared `_ensure_unit()` helper coerces raw floats into `pint` quantities at module entry points, guaranteeing dimensional correctness without burdening callers. Chapter and notebook code import from `mlsysim.physics` directly. The appendix (Table 10) lists representative function signatures.

The **Hardware** registry holds 45 devices across five deployment tiers (Cloud, Workstation, Edge, Mobile, Tiny), spanning nine orders of magnitude in peak throughput from milliwatt-class microcontrollers (`Hardware.Tiny.ESP32-S3`, 520 KiB on-chip SRAM) to wafer-scale engines (`Hardware.Cloud.Cerebras-CS3`, 44 GB on-wafer SRAM, 125 PFLOP/s). Beyond NVIDIA GPUs (T4 and V100 through B200 and the rack-scale GB200 NVL72), the registry includes AMD MI250X and MI300X, Intel Gaudi 2 and Gaudi 3, AWS Trainium 2, and the full Google TPU lineage (v1 through v6/Trillium), enabling cross-vendor design-space exploration, plus workstation (Apple M3 Max, DGX Spark), edge (Jetson, Coral), mobile (Apple A17 Pro, Snapdragon 8 Gen 3, Tensor G3), and TinyML (ESP32-S3, nRF52840, Himax WE-I Plus) devices. The `list(sort_by=)` class method enables programmatic comparison, and users extend any zoo by instantiating new typed objects (Listing 2) or adding YAML data files, ensuring custom entries

Table 4: **Physics Domain Modules**. Each module owns the analytical formulas for one domain. The two extension modules sit outside the wall taxonomy (Section 9).

Module	Domain	Key Formulas
performance	Node	Roofline bottleneck, Amdahl’s speedup, pipeline bubble
memory	Node	Model memory, activation memory, KV-cache, checkpoint size
serving	Node	Erlang-C M/M/c queue latency
communication	Fleet	Ring/tree/hierarchical AllReduce, All-to-All
reliability	Fleet	Young–Daly, cluster MTBF, failure probability
transformer	Algorithm	Training/decode FLOP counts
economics	Operations	Fleet TCO, egress cost
networking	Data	Speed-of-light latency
statistics	Analysis	PSI drift detection, sample sizing
agents	Extension	Trajectory reliability, test-time compute cost, multi-agent coordination
robotics	Extension	Sensor-to-actuator latency, safe stopping distance, action-chunk cadence

participate in the same dimensionally strict pipeline as vetted ones. Figure 4 maps the implementation structure behind this composition path: panel (a) shows package architecture (registries, physics modules, core utilities, and invariant checks), while panel (b) traces a concrete analysis from typed inputs through the resolver chain to the three-level `SystemEvaluation` scorecard. Section B gives a representative snapshot of the current registry surface.

3.3 The Type System

MLSYS-IM’s type system is built on `Pydantic BaseModel` classes with `pint Quantity` fields, providing both schema validation and dimensional enforcement at construction time. The same schemas validate the YAML registry data: every `*.yaml` entry is instantiated through its `pydantic` type at import, so a malformed unit string fails the import rather than surfacing later as a wrong number, and a missing provenance record fails the provenance audit. The composition hierarchy is deliberately shallow: `HardwareNode` aggregates `ComputeCore` and `MemoryHierarchy` as direct fields, not through

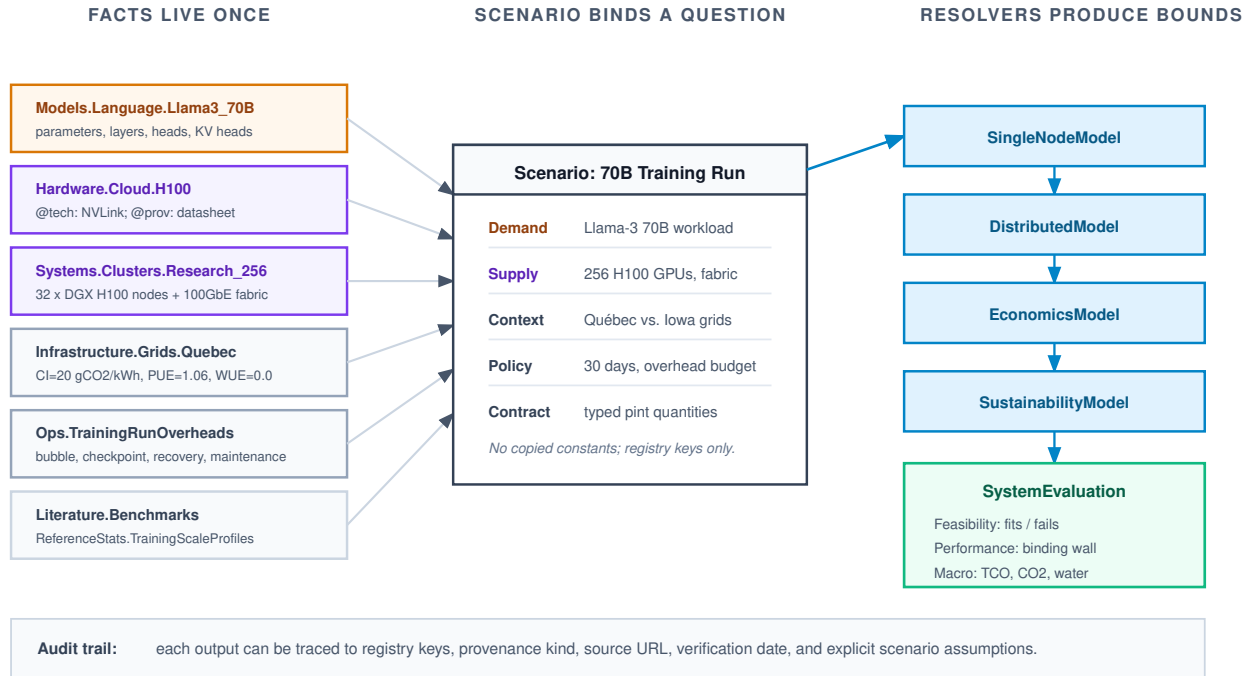


Figure 3: **Registry-Backed System Composition.** MLSYS · IM stores workload, hardware, systems, infrastructure, operational, and literature facts once, with typed schemas and provenance records. A scenario composes those reusable facts into a local infrastructure question—for example, training a 70B model on a 256-GPU H100 fleet in Québec or Iowa—and hands dimensioned objects to the resolver chain. The resulting `SystemEvaluation` can be traced back to the registry keys, source records, and scenario assumptions that produced it.

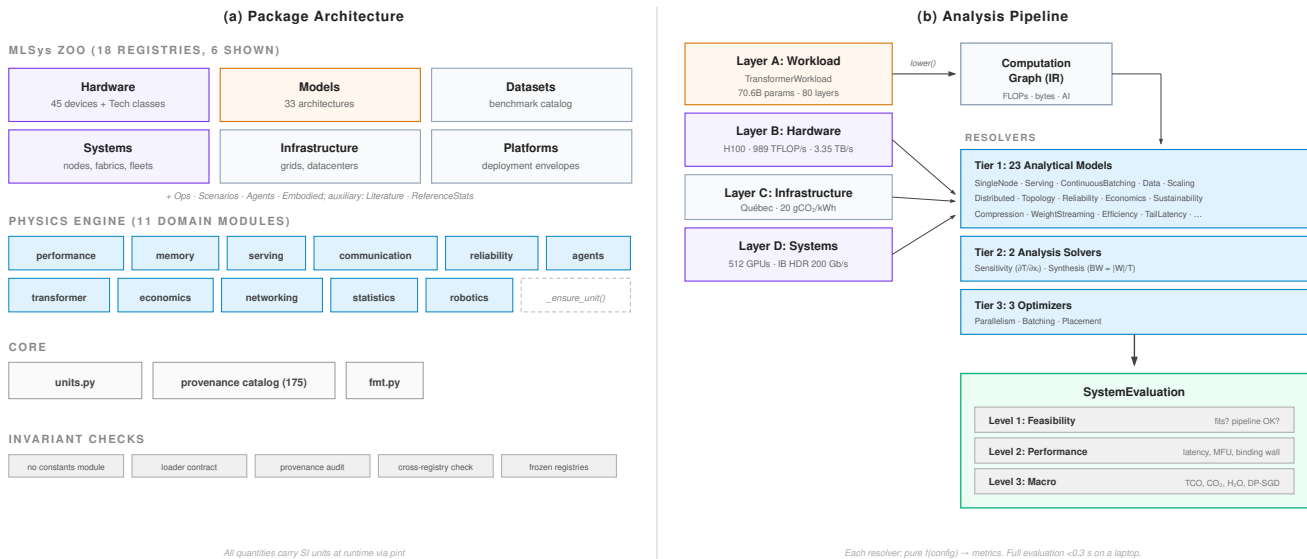


Figure 4: **System Anatomy.** (a) Package architecture: 18 curated registries (six shown) provide typed, provenance-tracked specifications, 11 domain physics modules supply the formulas, and five invariant checks enforce design decisions as the codebase evolves. (b) Analysis pipeline: a workload lowers to a hardware-agnostic intermediate representation; four input layers feed 28 stateless resolvers organized in three tiers; the output is a three-level `SystemEvaluation` scorecard (Feasibility, Performance, Macro).

deep inheritance. This design makes the relationship between a hardware specification and its physical quantities immediately legible:

```

1 from mlsysim.hardware.types import *
2 from mlsysim.core.units import Q_
3 node = HardwareNode(
4     name="Custom Accelerator",
5     release_year=2025,
6     compute=ComputeCore(
7         peak_flops=Q_("500 TFLOPs/s"),
8         precision_flops={"fp8": Q_("1000 TFLOPs/s")
9     }
10 ),
11     memory=MemoryHierarchy(
12         capacity=Q_("96 GiB"),
13         bandwidth=Q_("4 TB/s"),
14         tdp=Q_("500 W"))
15 print(node.ridge_point()) # → 125.0 flop/B

```

Listing 2: **Custom Hardware Node.** Composing a hardware specification with dimensional types.

The ComputationGraph IR bridges the demand-supply gap. When a solver calls `workload.lower()`, the workload computes its total operations, weight bytes, and arithmetic intensity, all in dimensioned quantities. For Mixture-of-Experts models, `SparseTransformerWorkload.lower()` uses *active* parameters for FLOPs but *total* parameters for memory footprint, correctly modeling the fundamental decoupling between compute cost and capacity requirements in sparse architectures (Shazeer et al., 2017).

The evaluation produces a `SystemEvaluation` scorecard that groups the metrics of the composed resolvers into Feasibility, Performance, and Macro levels. Students can inspect any individual level or view the aggregate to understand how constraints interact across the stack.

3.4 Extensibility

The layered architecture is designed for extension at every level. New workload types (e.g., a `RetrievalAugmentedWorkload` for RAG pipelines) require implementing the `lower()` method to produce a `ComputationGraph`. Resolvers that consume only the graph, such as the `SingleNodeModel`, then apply unchanged, while architecture-specific resolvers such as the `ServingModel` also require transformer fields. New hardware entries are added to the **Hardware** registry as declarative `HardwareNode` specifications (Listing 2), with no resolver changes needed. New resolvers can be introduced for emerging constraints by implementing the appropriate tier interface: a Tier 1 Model (e.g., a `PrivacyModel` for federated learning overhead), a Tier 2 Solver, or a Tier 3 Optimizer.

By accepting typed inputs and returning dimensioned outputs, the type system enforces correctness at every boundary, ensuring that custom extensions compose safely with existing components. This design ensures that `MLSYS·IM` can track the rapidly evolving ML systems landscape without requiring architectural changes to the core framework.

3.5 Public API and Presentation Layer

The package separates analysis from presentation. `from mlsysim import *` exposes the registries, the physics formulas, the unit vocabulary, and the formatting helpers; solver classes are imported through the stable `mlsysim.solvers` path, which mechanically re-exports the canonical implementations in `mlsysim.engine.solvers` so the two surfaces cannot drift. The `engine` package is the infrastructure-modeling core: the `Roofline` core (`engine.engine`), the 28 resolvers, the three-level evaluation, pipeline composition, design-space exploration, executable scenarios, the machine-readable wall taxonomy, calibration defaults, and empirical anchors. Presentation lives outside the engine: `mlsysim.fmt` provides a typed `fmt_*` formatter family (50+ helpers for quantities, counts, currencies, percentages, and scientific notation) whose outputs are Markdown-safe strings and whose precision-safety guards refuse to render non-finite values or to silently round a non-zero value to the literal “0”; `mlsysim.viz` supplies matplotlib plotters with a consistent style; and a command-line interface (`python -m mlsysim`) exposes the zoo browser, scenario evaluation, serving analysis, design-space optimization, a local-hardware Iron Law audit, and schema export.

4 Taxonomy of ML Systems Walls

The ML systems literature is rich with specialized models for specific bottlenecks, from the original Roofline model (Williams et al., 2009) and Chinchilla scaling laws (Hoffmann et al., 2022) to PagedAttention batching limits (Kwon et al., 2023) and datacenter sustainability accounting (Patterson et al., 2021). However, these constraints are typically studied in isolation. We synthesize this disjointed literature into a unified taxonomy of 20 distinct “Walls” plus 2 cross-cutting diagnostic tools (Sensitivity and Synthesis). We borrow the term from the computer architecture tradition (the “memory wall” (Wulf and McKee, 1995), the “power wall” (Hennessy et al., 2024)), where a *wall* denotes a hard physical or logical constraint that bounds system performance. Codifying these previously disparate equations into a single, composable framework, each wall maps to a

Table 5: **Notation.** Symbols used throughout; all quantities carry SI units at runtime via `pint`.

Symbol	Unit	Description
<i>Model & Workload</i>		
P, P_ℓ	params	Total / per-layer parameter count
$ W , W_\ell $	bytes	Bytes read per step / per layer
b_{prec}	B/param	Precision (e.g., 2 for FP16)
L, H, D	–	Layers, attention heads, head dim
S	tokens	Sequence length
B	samples	Batch size
K	–	Reasoning steps
C	FLOPs	Training compute ($6PD$)
I	FLOP/B	Arithmetic intensity
<i>Hardware & Infrastructure</i>		
Peak _{FLOPS}	FLOP/s	Peak accelerator throughput
BW_{HBM}	B/s	HBM bandwidth
BW_{inject}	B/s	Injection BW (wafer-scale)
BW_{link}	B/s	Per-link network bandwidth
N, G	–	Nodes in fleet, GPUs per node
α	s	Per-hop network latency
<i>Efficiency & Utilization</i>		
η	–	HW utilization ($\approx$ MFU)
η_{overlap}	–	Compute–comm overlap
ρ	–	Utilization ratio (queue/data)
$\beta, \beta_{\text{opt}}$	–	Bisection BW frac, optimizer multiplier
<i>Parallelism</i>		
TP, PP, DP, EP	–	Tensor, pipeline, data, expert parallel
V, M_{micro}	–	Virtual stages, microbatches
<i>Sustainability</i>		
PUE, WUE	–, L/kWh	Power / Water Usage Effectiveness
CI	gCO ₂ /kWh	Regional carbon intensity
<i>Key Derived Quantities</i>		
I^*	FLOP/B	Ridge point (Peak/ BW_{HBM})
B^*, P^*, D^*	varies	Optimal batch, model, dataset size
τ_{opt}	s	Optimal checkpoint interval

resolver (model or solver) that accepts typed inputs and produces dimensionally correct bounds. We are not aware of a prior tool that integrates these constraints in one executable engine. The complete taxonomy is summarized in Table 1.

The taxonomy proceeds outward from the silicon. Section 4.1 covers the seven Node walls that bound a single accelerator (compute, memory, software efficiency, serving, batching, streaming, tail latency). Section 4.2 adds the three Data walls that govern how samples reach the accelerator (ingestion, transformation, locality). Section 4.3 covers the three Algorithm walls that originate in the mathematics of learning (complexity, reasoning, fidelity). Section 4.4 adds the three Fleet walls that arise once a job spans multiple nodes (communication, fragility, multi-tenant scheduling). Section 4.5 covers the four Operations walls that determine whether a system should run at all (capital, sustainability, checkpoint, safety). Finally, Section 4.6 introduces the two cross-

cutting diagnostic tools (Sensitivity and Synthesis) that operate across the entire taxonomy.

4.1 Node (Single-Accelerator Resources)

The Node walls define what a single accelerator (one physical processing unit such as a GPU, TPU, or wafer-scale engine, used interchangeably with “individual accelerator”) can achieve in isolation, before any distributed coordination is considered. Multi-node analysis, where a server hosts several such accelerators on intra-node links such as NVLink, falls under the Fleet domain (Section 4.4). Node walls are the innermost constraints and the first a practitioner should evaluate.

Wall 1: The Compute Wall. Every accelerator has a hard throughput ceiling determined by the number of arithmetic units and the clock frequency. An H100, for example, provides a peak of 989 TFLOP/s at FP16 with Tensor Cores, establishing an upper bound that no software optimization can exceed. The `SingleNodeModel` evaluates the classical Roofline model bounds (Williams et al., 2009). If execution is entirely compute-bound, the minimum time to process the workload is dictated by the *Iron Law of ML Systems*:

$$T_{\text{compute}} = \frac{\text{OPs}}{\text{Peak}_{\text{FLOPS}} \times \eta} \quad (1)$$

where $\eta \in (0, 1]$ is the hardware utilization efficiency and OPs is the total operation count. Throughout this paper, η denotes the ratio of sustained to peak throughput; the related metric *Model FLOPS Utilization* (MFU) measures only model-useful FLOPs and excludes overhead such as activation recomputation. For first-order analysis, we treat $\eta \approx$ MFU; the distinction matters only when recomputation or non-model compute is significant. When this wall binds, the only remedy is faster silicon or fewer operations. **Assumptions:** Peak FLOPS is a hard ceiling; η is workload-dependent and must be specified or estimated from benchmark data.

Wall 2: The Memory Wall. High-bandwidth memory (HBM) imposes two ceilings: capacity (the model must fit) and bandwidth (weights must stream to compute units fast enough). An H100 reads HBM at 3.35 TB/s, yet its 989 TFLOP/s demand data at a rate that exceeds this bandwidth for any workload below ~ 295 flop/byte, making most LLM inference memory-bound, not compute-bound. During training, techniques like **Low-Rank Adaptation (LoRA)** and **Activation Recomputation** fundamentally alter the capacity constraint by trading compute or parameter trainability for drastically reduced memory footprints. The same `SingleNodeModel` computes (Williams et al., 2009):

$$T_{\text{memory}} = \frac{|W|}{BW_{\text{HBM}}} \quad (2)$$

where $|W|$ is the total bytes read per inference step. The realized execution time is the maximum of the two bounds:

$$T = \max(T_{\text{compute}}, T_{\text{memory}}) \quad (3)$$

The crossover between compute-bound and memory-bound regimes occurs at the *ridge point*, the arithmetic intensity at which the two ceilings intersect:

$$I^* = \frac{\text{Peak}_{\text{FLOPS}}}{BW_{\text{HBM}}} \quad (\text{flop/byte}) \quad (4)$$

Workloads with arithmetic intensity $I < I^*$ are memory-bound; those with $I > I^*$ are compute-bound. **Assumptions:** Peak FLOPS and HBM bandwidth are hard ceilings; MFU accounts for software inefficiency via η .

For training, capacity pressure is larger than inference weight residency alone. The `TrainingMemoryModel` exposes the first-order per-accelerator accounting directly:

$$M_{\text{train}} = M_{\text{weights}} + M_{\text{grad}} + M_{\text{opt}} + M_{\text{act}} + M_{\text{comm}}. \quad (5)$$

Mixed-precision Adam defaults to FP16/BF16 weights and gradients plus FP32 master weights and optimizer moments, while tensor, pipeline, expert, and ZeRO sharding reduce different terms of the sum (Shoeybi et al., 2019; Rajbhandari et al., 2020). Activation checkpointing reduces M_{act} by recomputing intermediate tensors during backward pass; the activation term implements the per-layer analytical bounds of Korthikanti et al. (2023) ($34sbh$ bytes per layer with selective recomputation; $34sbh + 5as^2b$ without, where s, b, h, a are sequence length, microbatch size, hidden dimension, and attention heads). The model intentionally reports each term separately so students can see whether a configuration fails because of optimizer state, activations, or communication buffers rather than a generic “out of memory” label.

Wall 3: The Software Wall. The gap between peak and achieved FLOP/s is typically larger than the gap between hardware generations. Megatron-LM’s single-V100 baseline sustained only 30% of theoretical peak FLOPs (Shoeybi et al., 2019), and published MFU for large training runs spans 21.3% to 46.2% (Chowdhery et al., 2023); much of the gap comes from memory traffic, idle kernels, and unfused operations. The `EfficiencyModel` models this as a multiplicative efficiency factor:

$$\eta = \frac{\text{FLOPS}_{\text{achieved}}}{\text{Peak}_{\text{FLOPS}}} \quad (6)$$

where $\eta \in (0, 1]$ modulates the Roofline ceiling, reducing the effective peak from $\text{Peak}_{\text{FLOPS}}$ to $\eta \times \text{Peak}_{\text{FLOPS}}$. FlashAttention (Dao et al., 2022), for example, reports a

$3\times$ end-to-end training speedup on GPT-2 at sequence length 1K by fusing attention into IO-aware kernels that cut HBM reads and writes; the `EfficiencyModel` represents this kind of gain by raising attention-layer η from 0.3 to an illustrative 0.75. When this wall binds, better kernels, not bigger chips, are the remedy. **Assumption:** η is a single scalar that aggregates all software inefficiencies; in practice, different operations (GEMM vs. attention vs. normalization) achieve different utilization on the same silicon. To avoid circularity when η is unknown, the `EfficiencyModel` draws its defaults from the registry’s literature bounds for large-scale training, $\eta = 0.30$ to 0.50 , consistent with published MFU (Chowdhery et al., 2023; Llama Team, AI Meta, 2024). At its default setting, attention layers get the lower bound, FFN layers the upper bound (capped at 0.60), and convolutions the midpoint (capped at 0.55). Students can use these defaults as starting points and refine as they gather profiling data.

Wall 4: The Serving Wall. Autoregressive LLM inference exhibits two distinct phases with fundamentally different Roofline characteristics (Pope et al., 2023). For a 70B model on two tensor-parallel H100s, a 512-token prefill takes ~ 73 ms (compute-bound) while each decode token costs ~ 21 ms (memory-bound). Prefill processes all input tokens in parallel (~ 7005 tokens/s), while decode generates one token per memory-bandwidth pass (~ 48 tokens/s), a $\sim 147\times$ throughput gap. The `ServingModel` decomposes end-to-end inference latency as:

$$T_{\text{prefill}} = \frac{2P \cdot S_{\text{in}}}{\text{Peak}_{\text{FLOPS}} \times \eta} \quad (\text{compute-bound}) \quad (7)$$

$$T_{\text{decode}} = \frac{|W|}{BW_{\text{HBM}}} \quad (\text{memory-bound}) \quad (8)$$

where S_{in} is the input sequence length, P is the parameter count, and $|W|$ includes both model weights and KV-cache reads (which grow with batch size and context length: $|W| = |W_{\text{model}}| + |W_{\text{KV}}|$). The $2PS$ term accounts for the linear projection FLOPs. The self-attention computation adds $\mathcal{O}(S^2)$ FLOPs per layer; under the `ServingModel` FLOP accounting it is 13% of prefill FLOPs for Llama-3 70B at 8,192 tokens and 38% at 32,768. The `ServingModel` includes this attention term explicitly. When serving multiple concurrent requests ($B > 1$), model weights are loaded from HBM once per step while KV-cache reads grow linearly with the batch: $T_{\text{step}} = (|W| + B \cdot |KV_1|)/BW$, where $|KV_1|$ is the per-request KV-cache size. Because each step produces B tokens simultaneously, the effective per-token cost is T_{step}/B , amortizing the weight read across requests. At large batch sizes the per-request KV cache dominates and throughput saturates.

The model also exposes an optional chunked-prefill estimate inspired by Sarathi-Serve (Agrawal et al., 2024a). When `prefill_chunk_tokens` is set, only the new, uncached prefill work is partitioned into chunks:

$$C_{\text{prefill}} = \max \left(1, \left\lceil \frac{S_{\text{in}} - S_{\text{cache}}}{S_{\text{chunk}}} \right\rceil \right), \quad (9)$$

$$T_{\text{stall}} = \max_i (T_{\text{chunk},i}) \quad (10)$$

where $T_{\text{chunk},i}$ is the compute time for chunk i plus one dispatch tax. The resulting `ServingResult` reports `prefill_chunks` (C_{prefill}), `prefill_chunk_time` (T_{stall}), and `decode_stall_bound`, which is set to T_{stall} as a first-order bound on how long the slowest prefill chunk can interfere with ongoing decode iterations. This is a coarse analytical bound, not a full scheduler: total TTFT still includes the same prefill compute work plus one dispatch tax per chunk, and the solver does not claim Sarathi-Serve’s stall-free packing behavior.

For deployment sizing, the serving-capacity resolver combines serving latency, continuous-batching capacity, and tail-queueing estimates. Given average generated length G and per-replica token throughput R_{tok} , it estimates request capacity as

$$Q_{\text{replica}} = \frac{R_{\text{tok}}}{G}, \quad (11)$$

$$P99_{\text{request}} \approx T_{\text{base}} + P99_{\text{queue}} \leq T_{\text{target}}. \quad (12)$$

This converts a common serving question—“how many replicas do I need for a QPS and P99 target?”—into a transparent composition of the latency, batching, and queueing walls. The compute efficiency η remains an exposed parameter with the same interpretation as in the Roofline model; the default is a starting point, not a claim of universal serving efficiency.

The prefill phase processes all input tokens in parallel and is compute-bound; the decode phase generates one token at a time and is memory-bandwidth-bound. The solver incorporates modern paradigms including **Prompt Caching** (prefix caching (Zheng et al., 2024), which reduces TTFT by skipping prefill for previously computed KV-cache entries), **Speculative Decoding** (probability-weighted verification using a smaller draft model (Leviathan et al., 2023)), **Disaggregated Serving** (phase splitting onto different hardware with KV-cache network transfer (Patel et al., 2024)), and **Chunked Prefill** (optional chunk-level stall bounds for interleaving prefill work with decode (Agrawal et al., 2024a)). This duality explains why batching strategies that improve prefill throughput may have no effect on decode latency, because the two phases are bound by different resources. **Assumptions:** Prefill is compute-bound for sequence

lengths $S_{\text{in}} \gg 1$; decode is memory-bandwidth-bound at batch size 1. Chunked prefill partitions the same upper-bound prefill work by token count with a fixed dispatch tax per chunk, so `decode_stall_bound` is a scheduling proxy rather than a measured tail-latency guarantee. At large batch sizes decode compute grows with the batch and can approach the compute ceiling. The `ServingModel` does not model this crossover, because its decode time is the memory-bound term alone; the `SingleNodeModel` Roofline shows which ceiling binds.

Wall 5: The Batching Wall. Decode is memory-bound, so the number of requests that share a decode step is the main throughput lever, and the KV-cache allocator decides how many requests fit. A static allocator reserves a contiguous slot of S_{max} tokens for every request, whatever length the request actually reaches. A paged allocator, as in PagedAttention (Kwon et al., 2023), divides the KV budget into blocks of p tokens and gives a request of S tokens $\lceil S/p \rceil$ blocks on demand. With M_{KV} the HBM left after the weights and k the KV bytes per token, the `ContinuousBatchingModel` computes the concurrency each allocator admits:

$$N_{\text{static}} = \left\lfloor \frac{M_{\text{KV}}}{k S_{\text{max}}} \right\rfloor, \quad N_{\text{paged}} = \left\lfloor \frac{\lfloor M_{\text{KV}} / (k p) \rfloor}{\mathbb{E}[\lceil S/p \rceil]} \right\rfloor \quad (13)$$

where request lengths are exponential with mean $\bar{S}$ and cut off at S_{max} , and the scheduler’s maximum batch size caps both counts. The unused share of each allocation is $1 - \bar{S}/S_{\text{max}}$ under static reservation and about one partially filled block per request under paging. Memory-bound decode reads the weights once per step plus the tokens that requests have filled, so $t_{\text{step}} = (|W| + N\bar{S}k)/BW_{\text{HBM}}$ and token throughput is N/t_{step} . For Llama-3 8B on an H100 with $S_{\text{max}} = 8,192$ and $\bar{S} = 2,048$, static reservation admits 65 requests and leaves 75% of each reservation unused, while 16-token pages admit 259 requests with 0.4% unused, a $1.56\times$ gain in decode throughput; 2,048-token pages admit only 165. Kwon et al. profile contiguous pre-allocation and find that only 20.4% to 38.2% of KV-cache memory holds token states, with the remainder lost to reserved slots, internal fragmentation, and external fragmentation, whereas vLLM reaches 96.3% (Kwon et al., 2023). **Assumptions:** Request lengths are exponential, and every admitted request is sized at its full length (peak occupancy), which makes paged capacity conservative. The static baseline is max-length reservation with no external fragmentation; exact-size and power-of-two contiguous allocators are not modeled. When the batch-size cap binds both allocators, allocation policy does not change throughput.

Wall 6: The Streaming Wall. Wafer-scale architectures (Lie, 2022) (e.g., Cerebras CS-3) invert the conventional memory hierarchy. Activations reside on-wafer in SRAM while model weights stream from external MemoryX nodes, shifting the bottleneck from HBM bandwidth to injection interconnect bandwidth. The `WeightStreamingModel` models this as:

$$T_{\text{layer}} = \max\left(\frac{|W_\ell|}{BW_{\text{inject}}}, \frac{2P_\ell \times B}{\text{Peak} \times \eta}\right) \quad (14)$$

where $|W_\ell|$ is the layer weight size in bytes, $P_\ell = |W_\ell|/b_{\text{prec}}$ is the parameter count (with b_{prec} bytes per element), and the factor of 2 accounts for the multiply-accumulate FLOPs per parameter. The two terms inside the max represent the injection time (weight delivery) and the compute time (matrix arithmetic) for one layer. When B is small, weight injection dominates and the compute engine sits idle; when B is large, compute dominates and the injection link sits idle. Setting the two terms equal and substituting $|W_\ell| = P_\ell \times b_{\text{prec}}$ yields the optimal batch size $B^* = (b_{\text{prec}} \times \text{Peak} \times \eta)/(2 \times BW_{\text{inject}})$. This result depends only on numerical precision, peak compute, and injection bandwidth, independent of layer size. This is the unique operating point where injection and compute perfectly overlap, maximizing utilization of both resources. **Assumptions:** Layer weights dominate the injection payload; 10% overhead is reserved for working memory; perfect within-layer pipelining is assumed.

Wall 7: The Tail Latency Wall. At scale, P99 latency governs user experience, not the median. Fan-out compounds the tail: if a request must collect responses from 100 servers whose 99th-percentile latency is one second, 63% of such requests take more than one second (Dean and Barroso, 2013). The `TailLatencyModel` models inference replicas as an M/M/c queue using the Erlang-C formula (Harchol-Balter, 2013):

$$\mathbb{P}[\text{wait}] = \frac{(c\rho)^c / c! \cdot (1 - \rho)^{-1}}{\sum_{k=0}^{c-1} (c\rho)^k / k! + (c\rho)^c / c! \cdot (1 - \rho)^{-1}} \quad (15)$$

where c is the number of replicas, $\rho = \lambda/(c\mu)$ is per-server utilization, and λ, μ are arrival and service rates. The implementation evaluates the Erlang-C sum in log space (a log-sum-exp over log-gamma terms), so deep replica pools do not overflow the factorials in Equation (15). P99 latency grows non-linearly as $\rho \rightarrow 1$: for a 10-replica deployment, P99 at 80% utilization is $\sim 3\times$ the service time; at 95%, it rises to $\sim 10\times$, making the distinction between the two the difference between meeting and violating a latency SLA. **Assumption:** M/M/c is a first-order estimate; bursty arrivals and heavy-tailed service times usually raise tail latency above it, and the

reported P99 adds the wait quantile to the *mean* service time. Stability depends only on ρ , so a deployment that is unstable under M/M/c is unstable under any traffic with the same mean rates. To bridge toward more realistic service-time distributions, the solver accepts a `service_time_cv` parameter (coefficient of variation); when $cv \neq 1$, a Kingman correction factor $(c_v^2 + 1)/2$ scales the wait time, capturing the heavier tails of non-exponential service times.

4.2 Data (Movement & Pipelines)

The Data walls govern how data moves to the accelerator. A node that is locally unconstrained can still starve if the data pipeline cannot keep pace.

Wall 8: The Ingestion Wall. Storage I/O must supply training samples at the rate the accelerator consumes them. This wall binds most often in vision tasks, where raw images are large, and data stalls of this kind are well documented in DNN training (Mohan et al., 2021). The `DataModel` takes the demanded read rate as an input and computes the demand-supply ratio:

$$\rho_{\text{data}} = \frac{BW_{\text{demand}}}{BW_{\text{supply}}} \quad (16)$$

When $\rho_{\text{data}} > 1$, the accelerator stalls waiting for data. BW_{demand} is the product of batch size, sample size, and step rate; BW_{supply} is the effective throughput of the storage subsystem after accounting for read amplification and caching. For LLM training on tokenized text, BW_{demand} is small compared with image, audio, and video workloads, which are where this wall usually binds. **Assumption:** Storage bandwidth is the bottleneck, not network I/O (single-node training).

Wall 9: The Transformation Wall. JPEG decoding, tokenization, and augmentation execute on CPU cores, not on the accelerator. When the CPU preprocessing pipeline cannot keep pace, the accelerator stalls even if storage bandwidth is abundant. In the scenario of Case S2 (Section 7), each CPU worker decodes and augments ~ 850 images/s, so 32 workers cap the pipeline at $\sim 27,200$ images/s, below what an 8-GPU DGX A100's compute can consume. The `TransformationModel` quantifies this bottleneck (Murray et al., 2021):

$$T_{\text{transform}} = \frac{B \cdot S_{\text{sample}}}{C_{\text{cpu}}} \quad (17)$$

where B is the batch size (in samples), S_{sample} is the size of one sample in bytes, and C_{cpu} is the aggregate CPU preprocessing throughput in bytes/s. For N_{workers} workers that each finish r_{worker} samples/s after all transformations (decode, augment, normalize), $C_{\text{cpu}} = N_{\text{workers}} \cdot r_{\text{worker}} \cdot S_{\text{sample}}$, so $T_{\text{transform}} = B/(N_{\text{workers}} \cdot r_{\text{worker}})$.

Assumption: Preprocessing is CPU-bound and scales linearly with worker count up to core saturation.

Wall 10: The Locality Wall. Network topology determines the effective bandwidth available between any two nodes in the cluster. The `TopologyModel` models this through the *bisection bandwidth fraction* β (Leiserson, 1985), which varies by topology: Fat-Tree provides full bisection ($\beta = 1.0$), Dragonfly (Kim et al., 2008) is modeled at an illustrative $\beta \approx 0.85$, and the ring and 3D torus fractions shrink with node count N ($\beta = 2/N$ and $\beta = 2N^{-1/3}$). The effective inter-node bandwidth is:

$$BW_{\text{eff}} = \frac{BW_{\text{link}} \times \beta}{\text{oversubscription}} \quad (18)$$

This wall becomes binding when collective communication patterns demand bandwidth that the topology cannot supply at scale. For example, a 27-node 3D torus with 400 Gb/s links has $\beta \approx 0.67$ and delivers ~ 267 Gb/s per node, and at 64 nodes β falls to 0.50 (200 Gb/s); a Fat-Tree with the same links provides the full 400 Gb/s ($\beta = 1.0$), a $1.5\times$ advantage at 27 nodes that grows with cluster size and compounds across multi-hop AllReduce patterns. We place the Locality Wall in the Data domain rather than Fleet because it models the *physical topology constraint* on data movement (bisection bandwidth, oversubscription), whereas the Communication Wall (Wall 14, Fleet) models the *algorithmic cost* of specific collectives (AllReduce, All-to-All) that run atop that topology. The two walls interact but address distinct levels of abstraction. **Assumption:** β is a static function of topology and node count; real networks may exhibit dynamic congestion not captured by this static model.

4.3 Algorithm (Scaling & Compression)

The Algorithm walls arise not from hardware but from the mathematics of learning itself. They determine how much computation a workload *requires*, independent of the silicon that executes it.

Wall 11: The Complexity Wall. Chinchilla scaling laws (Hoffmann et al., 2022) establish that training compute scales jointly with model size P and dataset size D : doubling the parameters requires approximately doubling the tokens to remain compute-optimal. The `ScalingModel` implements:

$$C = 6PD \quad (\text{total training FLOPs}) \quad (19)$$

$$D^* \approx 20P \quad (\text{compute-optimal tokens}) \quad (20)$$

$$P^* = \sqrt{\frac{C}{120}} \quad (\text{optimal model size for budget } C) \quad (21)$$

These relations allow students to reason backward from a compute budget to the largest model that can be trained optimally, or forward from a model size to

the minimum viable training cluster. For example, a budget of 10^{24} FLOPs yields $P^* \approx 91\text{B}$ parameters with $D^* \approx 1.8\text{T}$ tokens; Chinchilla (70B, 1.4T tokens) was trained near this optimal frontier (Hoffmann et al., 2022). **Assumption:** Scaling law coefficients are fitted to published training runs; the `ScalingModel` does not flag extrapolation beyond that range, so budgets far outside published runs deserve caution.

Wall 12: The Reasoning Wall. Inference-time compute scaling introduces a cost that grows linearly with the number of reasoning steps K . The `InferenceScalingModel` models this as (Snell et al., 2025):

$$T_{\text{reason}} = K \times T_{\text{step}}(P, S_{\text{context}}) \quad (22)$$

where T_{step} is the per-step latency, itself a function of model size P and context length S_{context} . In practice, K varies by strategy: best-of- N sampling uses $K = N$, and Snell et al. (2025) evaluate best-of- N , beam search against a process reward model, and sequential revisions at generation budgets of up to 256. A $K=8$ query spends $\sim 8\times$ the decode compute of a single-pass answer, fundamentally altering serving economics (see Case S1, Section 7). **Assumption:** The prompt is prefilled once and the K steps are decoded at a fixed context length, so the model ignores context growth across steps.

Wall 13: The Fidelity Wall. Compression trades model fidelity for efficiency. Quantization reduces precision while pruning removes weights entirely. The accuracy–efficiency frontier is task- and architecture-dependent. The `CompressionModel` quantifies the two primary mechanisms (Han et al., 2016; Gholami et al., 2021):

$$r_{\text{quant}} = \frac{b_{\text{base}}}{b_{\text{target}}} \quad (\text{e.g., } 16/4 = 4\times \text{ vs. FP16}) \quad (23)$$

$$r_{\text{prune}} = \frac{1}{1-s} \quad (\text{memory reduction at sparsity } s) \quad (24)$$

Here b_{base} is a configurable baseline precision and b_{target} is the quantized precision. The `CompressionModel` defaults to an FP32 baseline, so INT4 reports $8\times$ unless the caller passes `baseline_precision="fp16"`, which gives the $4\times$ reduction relative to the FP16 or BF16 weights that models are usually served from. Quantization reduces memory reads but not FLOPs, shifting the arithmetic intensity rightward on the Roofline by a factor of r_{quant} and potentially crossing the ridge point from memory-bound to compute-bound. For pruning, r_{prune} gives the *memory reduction* ratio; actual compute speedup depends on sparsity structure. Unstructured sparsity yields no acceleration on current GPUs, while 2:4 structured sparsity can roughly double throughput on accelerators with sparse tensor support; NVIDIA quotes its

H100 Tensor Core figures *with sparsity* (NVIDIA Corporation, 2023). Post-training quantization methods such as GPTQ (Frantar et al., 2023) and AWQ (Lin et al., 2024) demonstrate that 4-bit quantization can preserve most accuracy for large language models, making a $4\times$ reduction against FP16 weights a practical operating point. However, compression ratio alone does not determine inference speedup. Unstructured pruning reduces storage but yields no compute speedup on GPU GEMM kernels; only structured pruning and $N:M$ sparsity (e.g., 2:4 on NVIDIA Ampere+) accelerate hardware execution. The `CompressionModel` reports both `memory_savings_pct` and `inference_speedup` to capture this distinction. The accuracy impact Δ_{acc} is modeled with fixed heuristic curves, since the fidelity-compression frontier varies by architecture and task. **Assumptions:** Accuracy degradation follows heuristic step functions for quantization and exponentials for pruning (Section 9); pruning compute speedup requires structured sparsity with hardware support.

4.4 Fleet (Multi-Node Coordination)

The Fleet walls arise when systems scale beyond a single node, requiring coordination across multiple accelerators connected by network fabric.

Wall 14: The Communication Wall. Distributed training requires gradient synchronization across N nodes, and the cost of that synchronization grows with both message size and node count. The `DistributedModel` models the dominant collective operations. For Ring AllReduce (Patarasuk and Yuan, 2009) and its ZeRO/FSDP partitioned equivalents (Reduce-Scatter and All-Gather):

$$T_{\text{ring}} = \frac{2(N-1)}{N} \cdot \frac{M}{B_{\text{link}}} + 2(N-1) \cdot \alpha \quad (25)$$

where M is the message size, B_{link} is the per-link bandwidth, and α is the per-hop latency. Modern clusters use **hierarchical AllReduce** to exploit the bandwidth asymmetry between intra-node interconnect (e.g., NVLink at 450 GB/s per direction, quoted as 900 GB/s bidirectional) and inter-node fabric (e.g., InfiniBand NDR at 50 GB/s per port, a $9\times$ gap). The `DistributedModel` implements a two-level model:

$$T_{\text{hier}} = T_{\text{intra}}(BW_{\text{NVLink}}, G) + T_{\text{inter}}(BW_{\text{IB}}, N) \quad (26)$$

where G is GPUs per node and N is the node count. Each level applies the ring formula (Equation (25)) at its respective bandwidth, making the inter-node phase the dominant cost at scale. After the intra-node reduce-scatter, each lead GPU holds M/G of the reduced result; the inter-node phase therefore operates on M/G ,

not the full M , making this the bandwidth-critical optimization of hierarchical collective design. In practice, modern frameworks also use **Compute/Communication Overlap**, hiding network latency behind backward pass computation. The `DistributedModel` models this with an overlap efficiency parameter $\eta_{\text{overlap}} \in [0, 1]$ (default 0.85, an illustrative value), yielding an exposed communication cost of $(1 - \eta_{\text{overlap}}) \cdot T_{\text{comm}}$. **Gradient Accumulation** further reduces effective communication cost. When accumulating gradients over K_{acc} microsteps before synchronizing, the DP AllReduce cost is amortized by $1/K_{\text{acc}}$, trading latency for communication efficiency at large global batch sizes. Tensor parallelism partitions model weights across devices and requires two AllReduce operations per transformer layer on *activations* (not weights) in the forward pass and two more in the backward pass (Shoeybi et al., 2019). Each AllReduce exchanges a tensor of size $B \times S \times H \times b$ bytes, where B is batch size, S is sequence length, H is hidden dimension, and b is bytes per element. Weights are partitioned once and remain stationary. Pipeline parallelism (Narayanan et al., 2021) introduces a bubble overhead:

$$B_{\text{pipeline}} = \frac{P_{\text{stages}} - 1}{V \cdot M_{\text{micro}} + P_{\text{stages}} - 1} \quad (27)$$

where V is the number of virtual pipeline stages and M_{micro} is the number of microbatches. For Mixture-of-Experts All-to-All dispatch:

$$T_{\text{a2a}} = \frac{(N-1)}{N} \cdot \frac{M}{B_{\text{link}}} + (N-1) \cdot \alpha \quad (28)$$

where M is the routed activation payload. The `MoERoutingModel` maintains a first-order MoE abstraction where the total parameter count determines memory residency, the active parameter count determines the compute workload, and any hot-expert imbalance inflates the routed payload. With routing imbalance factor $\gamma \geq 1$:

$$E_{\text{active,eff}} = \min(E_{\text{total}}, \gamma \cdot E_{\text{top-k}}), \quad (29)$$

$$P_{\text{active,eff}} = P_{\text{active}} \cdot \frac{E_{\text{active,eff}}}{E_{\text{top-k}}}. \quad (30)$$

The same factor is available in `DistributedModel` as `moe_routing_imbalance_factor`, where it inflates expert-parallel traffic without introducing a router simulation. This follows the load-balancing motivation of sparsely-gated MoE and Switch-style routing systems (Shazeer et al., 2017; Fedus et al., 2022; Lepikhin et al., 2020).

Wall 15: The Fragility Wall. Component failures are inevitable at scale. If each node has a mean time between

failures of $\text{MTBF}_{\text{node}}$, then a cluster of N nodes has (Daly, 2006):

$$\text{MTBF}_{\text{cluster}} = \frac{\text{MTBF}_{\text{node}}}{N} \quad (31)$$

The probability of at least one failure during a training run of duration T is:

$$P(\geq 1 \text{ failure}) = 1 - e^{-T/\text{MTBF}_{\text{cluster}}} \quad (32)$$

The Young-Daly formula (Young, 1974; Daly, 2006) gives the optimal checkpoint interval:

$$\tau_{\text{opt}} = \sqrt{2\delta \cdot \text{MTBF}_{\text{cluster}}} \quad (33)$$

where δ is the time to write one checkpoint. The `ReliabilityModel` uses these relations to estimate the fraction of compute lost to checkpointing and recovery, reporting a `goodput_ratio` that captures the effective fraction of wall-clock time spent on useful forward/backward computation after subtracting checkpoint writes and failure recovery. **Assumption:** Failures are independent and exponentially distributed (memory-less). The goodput estimate omits the work redone since the last checkpoint after each failure, so it is optimistic.

Wall 16: The Multi-tenant Wall. Production GPU clusters are rarely dedicated to a single job. Shared clusters introduce queueing delays that grow hyperbolically as utilization approaches 1.0, creating a tension between maximizing hardware utilization and minimizing researcher wait times. The `OrchestrationModel` treats the whole cluster as a single server and models job wait times with the M/D/1 mean wait (Harchol-Balter, 2013):

$$T_{\text{wait}} = \frac{\rho}{2\mu(1-\rho)} \quad (34)$$

where $\rho = \lambda/\mu$ is the cluster utilization, λ is the job arrival rate, and μ is the service rate. As $\rho \rightarrow 1$, wait times diverge hyperbolically. A cluster at 90% utilization has $9.0\times$ the mean wait and $16.2\times$ the mean queue length of one at 50%. **Assumption:** Job durations are approximately deterministic, which is reasonable for large training runs with predictable step times.

4.5 Operations (Economics, Sustainability & Safety)

The Operations walls capture constraints that are not about *how fast* a system runs but *whether it should run at all*: economic viability, environmental impact, checkpoint overhead, and responsible deployment.

Wall 17: The Capital Wall. Performance analysis is incomplete without economic constraints. A 1,024-GPU H100 cluster costs $\sim \$25.6\text{M}$ in CapEx alone; amortized over 3 years, the hardware cost of a single 30-day training run is about $\$701\text{K}$ before adding electricity.

The `EconomicsModel` computes total cost of ownership (Barroso et al., 2018):

$$\text{TCO} = \text{CapEx} + \text{OpEx}_{\text{energy}} + \text{OpEx}_{\text{maint}} \quad (35)$$

where $\text{OpEx}_{\text{energy}} = E_{\text{total}} \times P_{\text{kWh}}$ converts total energy consumption to dollar cost at the regional electricity price. At scale, CapEx dominates. For that run, electricity at $\$0.12/\text{kWh}$ is 7.9% of the $\$876\text{K}$ total, making hardware utilization (MFU) the primary lever for cost efficiency. **Assumption:** Linear amortization over a 3–5 year hardware lifetime.

Wall 18: The Sustainability Wall. Regional grid carbon intensity varies by more than an order of magnitude, making geography a first-order systems design variable. The `SustainabilityModel` converts energy into environmental impact (Patterson et al., 2021):

$$E_{\text{total}} = E_{\text{IT}} \times \text{PUE} \quad (36)$$

$$\text{CO}_2 = E_{\text{total}} \times \text{CI}_{\text{region}} \quad (\text{gCO}_2/\text{kWh}) \quad (37)$$

$$\text{H}_2\text{O} = E_{\text{total}} \times \text{WUE} \quad (\text{L/kWh}) \quad (38)$$

where PUE is the power usage effectiveness of the datacenter, $\text{CI}_{\text{region}}$ is the carbon intensity of the local grid, and WUE is the water usage effectiveness. This model captures *operational* carbon only. For battery-powered consumer devices, *embodied* carbon from manufacturing often dominates, and Gupta et al. (2021) attribute roughly 75% of such devices' life-cycle emissions to manufacturing and about 20% to energy use. The `SustainabilityModel` accepts an optional `embodied_carbon_per_device` parameter for lifecycle analysis. **Assumption:** Grid carbon intensity is a static regional constant; temporal variation (e.g., renewable intermittency) is not modeled. Power is modeled as an idle floor of 30% of TDP plus a component that scales linearly with MFU. The floor is a modeling assumption; Barroso and Hölzle (2007) observed that even an energy-efficient server of that era drew about half of its peak power when nearly idle.

Wall 19: The Checkpoint Wall. Long-running training jobs must periodically save model state (weights and optimizer states) to persistent storage, incurring an I/O penalty that directly reduces effective MFU. Checkpoint writes are serial stalls in an otherwise parallel job, so their share of wall-clock time caps effective MFU. The `CheckpointModel` models this I/O burst penalty, a first-order version of the checkpoint cost that systems such as Check-N-Run (Eisenman et al., 2022) work to reduce:

$$\text{MFU}_{\text{penalty}} = \frac{T_{\text{write}}}{T_{\text{interval}}} = \frac{|W| \times \beta_{\text{opt}} / \text{BW}_{\text{storage}}}{T_{\text{ckpt_interval}}} \quad (39)$$

where $|W|$ is the model weight size in bytes, and β_{opt} is the optimizer state multiplier, the ratio of total checkpoint bytes to model weight bytes. For mixed-precision Adam with FP16 weights, $\beta_{\text{opt}} \approx 7$ (FP32 master weights at 4 bytes + FP32 momentum at 4 bytes + FP32 variance at 4 bytes, plus FP16 model weights at 2 bytes, totaling 14 bytes per parameter vs. 2 bytes for the FP16 model alone). Gradients are ephemeral and not checkpointed. For a 70B-parameter model, the checkpoint size is $71\text{B} \times 14\text{B/param} \approx 1\text{TB}$, making storage bandwidth the binding constraint during I/O bursts.

Wall 20: The Safety Wall. Privacy and fairness guarantees impose quantifiable computational overhead. The `ResponsibleEngineeringModel` models the cost of differential privacy via DP-SGD (Abadi et al., 2016), where the noise multiplier scales inversely with the privacy budget:

$$\sigma \propto \frac{1}{\epsilon} \quad (40)$$

Per-sample gradient clipping and noise addition slow DP-SGD training, and the solver models the slowdown as $T_{\text{DP}} = T_{\text{base}}(1 + c/\epsilon)$ with an illustrative coefficient $c = 2$, which gives $3\times$ at $\epsilon = 1$ and $21\times$ at $\epsilon = 0.1$; c is not fitted to published measurements. Fairness constraints impose a complementary data requirement. Sufficient representation of minority subgroups demands additional training data proportional to $O(1/p_{\text{min}})$, where p_{min} is the smallest subgroup prevalence. Unlike the other walls, Wall 20 is more qualitative than quantitative, with overhead depending heavily on task, model architecture, and privacy mechanism. The solver reports a single multiplier, and real overhead varies with the task and privacy mechanism. **Assumption:** Privacy budget ϵ is a hard constraint; the solver reports the compute multiplier, not the privacy guarantee.

4.6 Analysis (Cross-Cutting Diagnostics)

The preceding 20 walls each model a specific physical or logical constraint. The final two entries are *diagnostic tools* rather than walls in the strict sense. They operate *across* the taxonomy rather than within a single domain, providing analysis capabilities that span all walls. The Sensitivity tool identifies which wall is binding, and the Synthesis tool derives minimum hardware from SLA requirements.

Wall 21: The Sensitivity Wall. Optimization is effective only when directed at the binding constraint; improving a non-bottleneck parameter yields no measurable gain. The `SensitivitySolver` identifies the binding constraint by estimating the sensitivity of end-to-end latency to each hardware parameter:

$$\frac{\partial T}{\partial BW_{\text{mem}}}, \quad \frac{\partial T}{\partial \text{Peak}_{\text{FLOPS}}}, \quad \frac{\partial T}{\partial M_{\text{cap}}} \quad (41)$$

The parameter with the largest sensitivity is the binding constraint, that is, the single upgrade that would yield the greatest performance improvement. Each reported sensitivity is the normalized latency response $(T_{\text{perturbed}} - T_{\text{base}})/T_{\text{base}}$ to a 10% parameter upgrade. For Llama-3 8B decode on an A100, the solver reports a memory-bandwidth sensitivity of -0.088 (a 10% bandwidth upgrade cuts latency by 8.8%) versus exactly zero for peak FLOPS, because under the hard max of Equation (3) a strictly memory-bound workload does not respond to a compute upgrade at all. This transforms “where should I invest?” from intuition into calculation. **Assumption:** Finite-difference approximation with 10% perturbation; second-order effects are ignored. The binding constraint is identified as the parameter with the largest absolute sensitivity.

Wall 22: The Synthesis Wall. The `SynthesisSolver` addresses the inverse problem: given a service-level objective (e.g., 50 ms inter-token latency), it inverts the Roofline bounds (Williams et al., 2009) to derive the minimum hardware specifications required to satisfy it:

$$BW_{\text{required}} = \frac{|W|}{T_{\text{target}}} \quad (42)$$

$$\text{FLOPS}_{\text{required}} = \frac{\text{OPs}}{T_{\text{target}} \times \eta} \quad (43)$$

This enables hardware-software co-design. Engineers specify an SLA and the solver derives the minimum hardware that satisfies it. For Llama-3 70B at a 50 ms inter-token latency target, the solver yields $BW_{\text{required}} = 141.2\text{ GB}/50\text{ ms} \approx 2,824\text{ GB/s}$, $1.38\times$ the A100’s 2.04 TB/s, confirming that this SLA requires at least 2 tensor-parallel A100s. **Assumption:** Hardware parameters are independently adjustable; co-design coupling between FLOPS and bandwidth is not modeled.

5 The 3-Tier Resolver Architecture

The walls define *what* constrains a system. This section formalizes *how* resolvers compose to produce end-to-end system evaluations. To clarify the mathematical intent of each component, MLSYS·IM organizes its analytical tools into a strict 3-tier taxonomy, summarized below before Section 5.1 shows how the tiers chain into multi-stage analyses through a shared, dimensionally typed contract, and Section 5.2 describes the three-level `SystemEvaluation` scorecard (Feasibility, Performance, Macro) that the framework returns.

Tier 1: Analytical Models (the “physics engine”). Models perform forward evaluation $Y = f(X)$ to determine the physical and logical consequences of a specific

system configuration. For example, the `ServingModel` estimates the time-to-first-token for a given LLM and GPU pair. Models are purely deterministic and make no decisions; they comprise the first 23 resolvers in our taxonomy.

Tier 2: Analysis Solvers (the “math engine”). Solvers perform algebraic inversion, $X = f^{-1}(Y)$, or finite-difference sensitivity analysis to find the parameter required to hit a specific target or the parameter that binds. For example, the `SynthesisSolver` takes a target latency SLA and works backward to derive the minimum memory bandwidth required.

Tier 3: Optimizers (the “engineering engine”). Optimizers perform constrained design-space search, $\max f(X)$ subject to $g(X) \leq c$, to find the best configuration among many valid options. Unlike Models and Solvers, which map directly to individual walls, Optimizers operate across the entire taxonomy. The `ParallelismOptimizer` sweeps power-of-two tensor and pipeline degrees (TP capped at the node size by default, with data parallelism filling the remainder) to maximize Model FLOPs Utilization (MFU) on a given cluster; the `BatchingOptimizer` searches for the maximum batch size that satisfies a P99 queueing latency SLA.

5.1 Stateless Composition and Chaining

Every resolver in `MLSYS·IM` is a pure function: it accepts a typed configuration, performs analytical computation, and returns a typed result object (a `Pydantic BaseModel` with dimensioned fields). Resolvers maintain no hidden state between invocations. Because they share a common type system, the output of one tier feeds naturally into the next, as illustrated in Figure 5. The reader should trace, in that figure, how the demand, supply, and topology layers feed a resolver chain that culminates in the three-level scorecard described in Section 5.2; the typed contract at each arrow lets the chain compose, and `pint` rejects any dimensional mismatch at run time. A full-stack analysis composes resolvers in sequence to resolve complex design questions. For example, determining the financial cost of training an optimally-sized model on a frontier cluster requires chaining algorithmic scaling, distributed execution, and macro-economics:

$$\text{Scaling} \xrightarrow{\mathcal{R}_1} \text{Distributed} \xrightarrow{\mathcal{R}_2} \text{Economics} \quad (44)$$

Listing 3 demonstrates this exact chain in `MLSYS·IM`. The `ScalingModel` calculates the optimal model size

```

1 import mlsysim
2 from mlsysim.solvers import ScalingModel,
   DistributedModel, EconomicsModel
3 from mlsysim.models.types import
   TransformerWorkload
4
5 # 1. Algorithm: Find optimal parameters for a fixed
   compute budget
6 budget = mlsysim.Q_("4e24 flop") # ~100K H100-days
   at 50% MFU
7 optimal = ScalingModel().solve(compute_budget=
   budget) # P* ~ 183B
   params
8
9 # Instantiate the demand (Layer A: Workload)
10 model = TransformerWorkload(
11     name="Frontier-Model", architecture="
   transformer",
12     parameters=optimal.optimal_parameters,
13     layers=80, hidden_dim=8192, heads=64
14 )
15
16 # 2. Fleet: Evaluate on a massive 8K GPU cluster (
   Layer D: Supply/Topology)
17 fleet = mlsysim.Systems.Clusters.Frontier_8K
18 seq_len = 4096
19 perf = DistributedModel().solve(
20     model, fleet,
21     batch_size=4096, tp_size=8, pp_size=4, seq_len=
   seq_len,
22     microbatch_count=16, # one sequence per
   microbatch
23 )
24
25 # 3. The Capital: Calculate TCO for the resulting
   training time
26 n_steps = optimal.optimal_tokens.to("count").
   magnitude / (4096 * seq_len)
27 duration = perf.step_latency_total * n_steps
28 tco = EconomicsModel().solve(fleet, duration_days=
   duration.to("day").magnitude)
29
30 print(f"Scaling Efficiency: {perf.
   scaling_efficiency:.1%}") # 73.4%
31 print(f"Training Duration: {duration.to('day'):.1f
   }") # 15.6
   day
32 print(f"Total Job Cost: ${tco.tco_usd:,.0f}")
   # $3,637,644

```

Listing 3: **Solver Composition.** Bridging algorithmic scaling, distributed execution, and fleet economics in a single executable chain.

for a given compute budget and hands that workload to the `DistributedModel` ($\mathcal{R}_1$), which computes the execution time on an 8,192-GPU fleet, factoring in 3D parallelism overhead. The `EconomicsModel` then converts that execution time into a Total Cost of Ownership ($\mathcal{R}_2$); a `SustainabilityModel` call can extend the same chain to carbon.

Each link in the chain preserves dimensional correctness. Units propagate through the computation, and

any mismatch raises an immediate error rather than producing a silently wrong result.

5.2 The SystemEvaluation Scorecard

MLSYS·IM provides a `Scenario.evaluate()` entry point that orchestrates solver composition automatically through a three-level evaluation. A scenario is a concrete executable case, such as a doorbell workload on a microcontroller or a frontier training workload on a fleet; sourced statistics used only as reference anchors remain outside the executable path in `ReferenceStats.*`.

Level 1: Feasibility. Does the model fit? Can the data pipeline keep pace? The framework checks memory capacity against model size, ingestion bandwidth against training throughput, and reports any wall where demand exceeds supply.

Level 2: Performance. What are the achievable latency, throughput, and utilization? The Roofline analysis (Equation (3)), communication modeling (Equation (25)), and pipeline bubble (Equation (27)) combine to produce end-to-end training step time.

Level 3: Macro. What does it cost, and what does it emit? The `EconomicsModel` estimates one year of TCO (Equation (35)) and carbon (Equation (36)) for the scenario’s fleet.

All three levels are computed and reported, so a Level 1 failure appears alongside the performance and cost figures it invalidates, while `Scenario.validate()` raises on the first failing level. Level 1 compares weight bytes against one accelerator’s capacity and checks declared data rates with the `DataModel`. Reading the levels in order reflects the dependency structure, since communication optimization is irrelevant if the model exceeds available memory. The complete implementation details and key assumptions for each solver are documented alongside their respective walls in Section 4.

6 Validation

An analytical framework earns trust through transparent confrontation with empirical ground truth. We validate MLSYS·IM along two axes: *accuracy* against published benchmarks, and the *speed* of design-space exploration. Our goal in validation is not to “curve-fit” the model to perfectly match empirical results by introducing arbitrary magic numbers. Rather, it is to demonstrate that a first-principles application of physics and generalized efficiency constants places the prediction squarely in the correct ballpark. As we argue in Section 9, a model that is off by 10% because it structurally ignores L2 cache misses is functioning exactly as intended, trading cycle-

level precision for architectural intuition. The anchors below illustrate this philosophy.

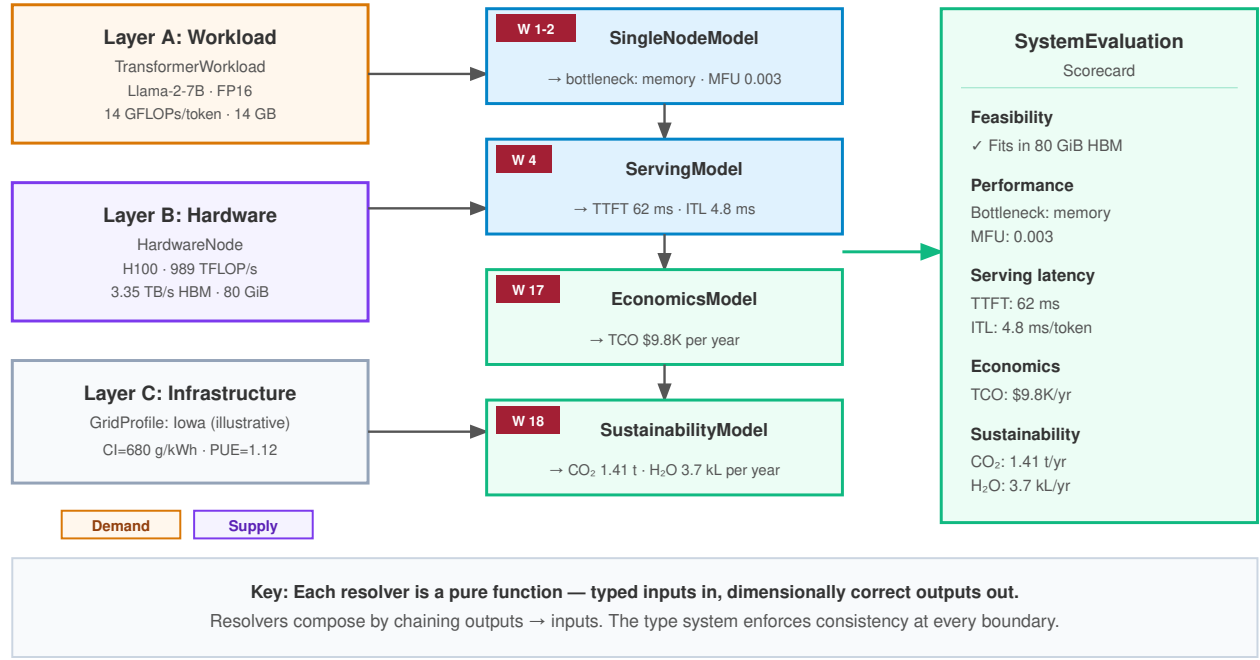
The remainder of this section presents the validation in four parts. Section 6.1 compares MLSYS·IM predictions with seven published results spanning single-node training, distributed training, inference, scaling laws, sustainability, and automated parallelism search, and reports each error. Section 6.2 reports design-space exploration speed. Section 6.3 discusses the accuracy scope and the empirical role of the efficiency parameter η . Finally, Section 6.4 argues that runtime dimensional strictness is itself a structural form of validation that numerical accuracy alone cannot supply.

6.1 Empirical Anchors

We compare MLSYS·IM predictions with seven published results. Every configuration and number below is computed by the committed generator script (`paper/scripts/generate_paper_values.py`), and `paper/scripts/validate_anchors.py` reruns every anchor, reports the error of each comparison with a published measurement, and fails if an identity check or the parallelism search no longer holds. Where an anchor needs the efficiency parameter η , we state the value used. Those values are chosen per workload class rather than measured, so these anchors test whether one utilization factor places a workload in the right regime and magnitude, not whether the model predicts throughput without calibration.

Anchor 1: ResNet-50 on DGX A100 (Single-Node Training). For ResNet-50 training on a DGX A100 node (8× A100 GPUs) at a global batch size of 2,048 (256 per GPU), MLSYS·IM’s `SingleNodeModel` with end-to-end utilization $\eta = 0.19$ predicts 2,376 samples/s per GPU, or 19,007 samples/s across the node under ideal intra-node data parallelism, in the compute-bound regime. NVIDIA reports 17,400 images/s for this node and per-GPU batch size with mixed precision and XLA (NVIDIA Corporation, 2020), so the prediction is 9.2% over the measurement. The low η reflects that ResNet-50’s small convolution kernels, data pipeline, and framework overhead cannot saturate the A100’s tensor cores the way large GEMM-dominated Transformer layers can.

Anchor 2: Llama-2-70B Decode on H100 (Inference Floor). For autoregressive decoding of Llama-2-70B (FP16, batch size 1), the weights total 140 GB, requiring at least 2 tensor-parallel H100s. MLSYS·IM’s first-order estimate divides total weights by aggregate bandwidth, $140 \text{ GB} / (2 \times 3.35 \text{ TB/s}) = 20.9 \text{ ms}$ per token. This is a memory-bandwidth floor that excludes KV-cache reads, NVLink synchronization, and scheduler overhead. We



Example: Llama-2-7B, batch 1, 2,048-token context, one H100 serving for a year on the Iowa grid profile.

Figure 5: **Resolver Composition.** Three input layers feed four resolvers. Each resolver is a pure function: typed inputs in, dimensionally correct outputs out. The scorecard aggregates three evaluation levels: Feasibility, Performance, and Macro (economics, sustainability, safety).

report it without an error because we have not found a published measurement for this exact configuration; it stays in the table as the inference-side regime check.

Anchor 3: Llama 3 Training at 16K H100s (Distributed Training). Meta’s Llama 3 report (Llama Team, AI Meta, 2024) documents 41% BF16 MFU for 405B pre-training on 16,384 H100 GPUs with TP=8, PP=16, and DP=128 at an 8,192-token sequence length, on a 400 Gb/s RoCE fabric, with fully sharded data parallelism for optimizer states and gradients and without activation checkpointing. We configure MLSYS·IM’s DistributedModel with that layout (TP=8, PP=16, DP=128, ZeRO stage 2), 2,048 sequences of 8,192 tokens per step, which leaves one sequence for each of at most 16 pipeline microbatches per DP group, the registry’s 400 Gb/s InfiniBand NDR fabric in place of RoCE, compute–communication overlap $\eta_{\text{overlap}} = 0.85$, and $\eta = 0.42$. It predicts an aggregate MFU of 20.6%, 49.6% below the reported value (20.4 percentage points). The miss is structural. With 16 microbatches the 1F1B bubble adds 5.6 s of idle time to 6.0 s of compute, 46.1% of the modeled step. Meta instead runs an interleaved schedule that divides the bubble by the number of virtual stages per pipeline rank, which the report does not give, and frees stage tensors as soon as they are no longer needed. Without those two optimizations MLSYS·IM’s

activation accounting also places this layout at 88.3 GB per GPU, above the 85.9 GB of an H100, although Meta trains it without checkpointing. The anchor therefore marks the limit of a plain 1F1B pipeline model rather than testing η .

Anchor 4: PaLM Model FLOPs Utilization (Pod-Scale Training). Google’s PaLM-540B (Chowdhery et al., 2023) was trained without pipeline parallelism on 6,144 TPU v4 chips in two pods of 3,072 chips. Within each pod it used 12-way model parallelism and 256-way fully sharded data parallelism, across the two pods it used data parallelism over the datacenter network, and it rematerialized activations, achieving 46.2% MFU and 57.8% hardware FLOPs utilization; the difference is the rematerialization FLOPs that hardware utilization counts and MFU excludes. We model each pod as one ICI domain at the TPU v4’s 300 GB/s per direction (Jouppi et al., 2023) and the cross-pod link at 1.65 GB/s per chip, which spreads PaLM’s reported aggregate burst of 81 Tbps over all chips and is therefore a floor on capacity. With TP=12, DP=512, ZeRO stage 3, full recomputation, a global batch of 2,048 sequences of 2,048 tokens, TPU v4 compute of 275 TFLOP/s, and $\eta = 0.47$, each chip holds 4.4 GB of training state, the scaling efficiency is 95.8%, and the predicted aggregate MFU is 33.8%, 26.9% below the reported value. Com-

munication costs little in this layout, so the gap sits mostly in η . Recomputation leaves each replica’s model FLOPs utilization at 35.2%, while PaLM’s 57.8% hardware utilization implies a kernel efficiency well above the $\eta = 0.47$ we assume.

Anchor 5: Chinchilla Scaling Law (Implementation Consistency). The Chinchilla paper (Hoffmann et al., 2022) establishes that compute-optimal training uses about 20 tokens per parameter. `MLSYS·IM’s ScalingModel` uses the compute approximation $C = 6PD$ and derives the optimal allocation $P^* = \sqrt{C/120}$. For Chinchilla’s own budget, $C = 6 \times 70B \times 1.4T \approx 5.88 \times 10^{23}$ FLOPs, the solver returns $P^* = 70.0B$ and $D^* = 1.40T$. This is an implementation consistency check rather than an independent validation, because Chinchilla’s 1.4T tokens are exactly 20 times its 70B parameters, so P^* returns 70B by construction.

Anchor 6: Training Carbon Footprint (Unit Consistency). Patterson et al. (2021) estimate that training GPT-3 on V100 GPUs consumed 1,287 MWh and emitted 552 tonnes CO₂e using the 2020 US average grid intensity (429 gCO₂e/kWh). `MLSYS·IM’s carbon_from_energy` reproduces 552.1 tonnes from those inputs. Since the published figure is itself this product, the anchor checks unit handling, not emissions accuracy.

Anchor 7: Llama 3 Parallelism Strategy (Optimizer Search). To exercise the Tier 3 design-space search, we configure the `ParallelismOptimizer` with the Llama 3 405B model, its 16,384-GPU H100 cluster (Llama Team, AI Meta, 2024), and Meta’s 8,192-token sequences. The optimizer sweeps power-of-two TP and PP degrees, pre-screens each candidate so that the per-GPU shard of weights, gradients, and Adam optimizer state fits in 90% of HBM, prices each survivor with the `DistributedModel`, drops any whose training state no longer fits once activations are counted, and ranks the 14 remaining candidates by MFU. Because `MLSYS·IM’s` activation accounting admits no split at this sequence length without recomputation (Anchor 3), the search runs with full activation recomputation. It selects TP=8, PP=16, DP=128, the TP=8, CP=1, PP=16, DP=128 layout Meta reports for this stage; Meta uses context parallelism only in its 131K-token stage. The match is narrow, since the winning split beats the runner-up (TP=8, PP=32) by only 1.4% in MFU. It does not depend on the optimizer’s default cap of TP at the node size, because raising the cap to 64 returns the same TP=8, PP=16 split.

The seven anchors exercise the Node, Algorithm, Fleet, and Operations domains and both Roofline regimes; the Data and Analysis domains have no published anchor yet. Table 6 summarizes the results.

Registry entries record `metadata.provenance` (Section 3.1.1). Datasheet, literature, and industry-report numbers require a verified URL and date, estimates and derived values require documented justification, and the provenance audit tool (`mlsysim.tools.audit_provenance`) fails on any value in the core registries whose provenance is missing or weak, which keeps “magic numbers” from accumulating silently as the framework evolves.

6.2 Design-Space Exploration Speed

On an Apple M5 Max laptop (Python 3.11.15), `MLSYS·IM’s SingleNodeModel` evaluates 1,000 hardware-model-batch configurations in 517ms (median of five warm runs), about 517 μ s per configuration. For comparison, ASTRA-sim 2.0 reports 21.42 minutes to simulate a single 1MB AllReduce on 64 NPUs with its cycle-level network backend and 1.70 seconds with its analytical backend (Won et al., 2023). These measurements are not like-for-like, since ASTRA-sim models collective communication at much higher fidelity, but they place `MLSYS·IM’s` per-configuration cost several orders of magnitude below cycle-level simulation. Sub-second execution enables interactive parametric sweeps (e.g., varying HBM bandwidth, substituting fat-tree for torus topology, or relocating the datacenter from Iowa to Québec) that would be impractical with cycle-level simulation.

6.3 Accuracy Scope and Limitations

`MLSYS·IM` provides first-order estimates, not cycle-accurate predictions. We do not re-derive the resolver abstraction here. It is the same first-principles structural bounds plus a single efficiency η for unmodeled second-order effects, as in Sections 1 and 3.1. **Calibrating η for validation.** η is the one free parameter in the throughput and MFU anchors, and we select it per workload class ($\eta = 0.19$ for ResNet-50 training on A100, 0.42 for Llama 3 405B, and 0.47 for PaLM). These values are chosen per workload class, neither fitted to the reported results nor measured, so those anchors test whether one utilization factor combined with first-principles memory and communication models lands in the correct regime and magnitude. The Roofline parallel (peak versus achievable ceilings, and which constraint binds) is stated in Section 1; see Williams et al. (2009). Anchors 5 and 6 need no efficiency parameter, but both reproduce the arithmetic that defines the published value, so they check implementation consistency rather than physical accuracy. Section 9 details the specific phenomena this abstraction cannot capture and the resulting accuracy boundaries.

Table 6: **Validation Summary.** Predicted versus reported values for the seven anchors. Error is $|(\text{pred.} - \text{rep.}) / \text{rep.}|$. Anchors 5 and 6 reproduce the arithmetic that defines the published value and are consistency checks. Every predicted value is regenerated from the released package.

Anchor	Predicted	Reported	Error
1: ResNet-50 DGX A100	19,007 samples/s	17,400 images/s	9.2%
2: Llama-2 70B decode floor (TP=2)	20.9 ms/token	not measured	–
3: Llama 3 405B MFU	20.6%	41%	49.6%
4: PaLM-540B MFU	33.8%	46.2%	26.9%
5: Chinchilla P^*	70.0B	70B	identity
6: GPT-3 CO ₂ e	552.1 t	552 t	identity
7: Llama 3 parallelism	TP=8, PP=16, DP=128	TP=8, PP=16, DP=128	exact match

For `MLSYS·IM`’s intended use cases (architectural reasoning, lab exercises, capacity planning, and design-space exploration), first-order accuracy is sufficient and often preferable. A student who understands *why* a system is memory-bound has learned more than one who can predict its throughput to three decimal places.

To safeguard against drift, a separate set of representative predictions (`engine.empirical: ResNet-50` training throughput on A100 and H100, Llama-3-8B decode latency, and GPT-3 training FLOPs) is bound to sourced tolerance bands in the test suite, and `paper/scripts/validate_anchors.py` recomputes the anchors in this section, reports the error of each published comparison, and fails when an identity check or the parallelism search no longer holds.

6.4 Dimensional Correctness as Validation

Beyond numerical accuracy, dimensional strictness (Section 3.1.2) provides a structural form of validation. Unit mismatches raise immediate errors rather than producing silently wrong results, eliminating a category of bugs that numerical validation alone cannot catch.

6.5 Design Invariant Enforcement

Dimensional strictness (Section 3.1.2) catches unit errors at runtime. A complementary question is whether the framework’s *architectural* invariants hold as the codebase evolves: do all hardware specifications flow through typed registries? Does every formula carry `pint` units end-to-end? Can a contributor accidentally reintroduce a bare constant that bypasses the registry? `MLSYS·IM` answers these with five machine-checked structural invariants, enforced by a test suite of 2,261 tests, each encoding a design decision that would otherwise erode through incremental changes:

1. **Specification separation.** There is no constants module at all: the legacy `core/constants.py` was deleted outright once its last value migrated, and a CI gate now pins its *absence* (any attempt to reintroduce

the module, or a compatibility shim for it, fails). Every domain figure lives in its category registry; measurement units live in `core.units`.

2. **Loader contract.** The YAML registry loader rejects duplicate keys outright and requires every `@tech:` and `@prov:` reference to resolve, so a typo in a data file fails at import rather than silently shadowing or orphaning a specification.
3. **Provenance coverage.** A dedicated audit (`mlsysim.tools.audit_provenance`, also run as a test) walks every registry leaf and fails on missing or weak provenance, enforcing the kind-specific evidence rules of Section 3.1.1 across the core registries.
4. **Cross-registry consistency.** When the same physical quantity appears in multiple registries (e.g., NVLink bandwidth in both **Hardware** and **Systems** node definitions), a dedicated check verifies that the DGX node definitions agree with their accelerator entries. This prevents silent divergence between overlapping specification surfaces.
5. **Registry immutability.** Every registry entry is a frozen `pydantic` model: attribute assignment raises at the call site. Entries are shared singletons read by every notebook cell in a rendering process, so without this guarantee a single stray write (`H100.tdp = ...`) would silently corrupt every downstream calculation; with it, the mistake fails loudly where it happens. Code that needs a perturbed variant builds an explicit copy (`model_copy`).

Provenance and loader checks apply to every new registry entry automatically, while the cross-registry check covers the DGX node definitions. Empirical-anchor tests (`engine.empirical`) complete the picture by binding representative solver outputs to sourced benchmark envelopes. The effect is that the zoo architecture described in Section 3.2 is not merely documented but *enforced*, so

the framework’s design decisions survive community contributions and long-term evolution.

7 Usage & Case Studies

MLSYS·IM is designed for three audiences: students developing quantitative reasoning skills, instructors preparing demonstrations, and researchers evaluating design trade-offs. We present two representative use cases per persona, each illustrating how solvers compose to answer questions that span multiple walls. For a cookbook of runnable code examples demonstrating how users define hardware, perform algebraic design-space sweeps, and invoke SLA-driven hardware synthesis programmatically in Python, see the Appendix (Listings 8, 9, 10, 11, 12, 13, 14, 15, and 16).

7.1 Student Use Cases

Students interact with MLSYS·IM primarily through single-solver queries, short resolver chains, and browser-friendly exercises in the companion textbook. Students can manipulate hardware parameters (e.g., batch size, SLA targets, carbon taxes) and observe how binding constraints shift without needing a backend server or physical hardware.

We present two examples chosen to illustrate complementary aspects of the framework. The first (S1) demonstrates *vertical* resolver composition, chaining inference and economics models to connect an algorithmic decision (chain-of-thought reasoning) to its infrastructure cost. The second (S2) demonstrates *horizontal* composition, combining data-pipeline and compute models to diagnose a bottleneck that moves from the accelerators to the CPU input pipeline.

7.1.1 S1: Chain-of-Thought Cost

A student investigates the inference economics of chain-of-thought (CoT) prompting. Using Llama-3 70B on H100 hardware, they configure the `InferenceScalingModel` with $K=8$ reasoning steps. Each step generates ~ 50 tokens (the calibrated default) at the memory-bound decode rate, so the total reasoning time is $T_{\text{reason}} = \text{TTFT} + K \cdot 50 \cdot \text{ITL}$. The solver reports a total reasoning time of ~ 17.9 s, a $6.5\times$ per-query *latency* multiplier relative to a single-step answer (the one-time prefill TTFT amortizes across the K steps). The *cost* multiplier is larger and tracks total work rather than latency: CoT generates $8\times$ as many decode tokens ($K \cdot 50$ vs. 50), and because decode is memory-bandwidth-bound the serving throughput is set by token volume. Feeding this into the `EconomicsModel`, a deployment sized for 100 QPS of single-pass queries must grow its H100 fleet, and hence its TCO, by $\sim 8\times$ to sustain the same query rate under

CoT. The two multipliers are distinct: MLSYS·IM shows chain-of-thought taxes both latency and infrastructure cost, but by different factors.

7.1.2 S2: CPU Pipeline Bottleneck

A student configures ResNet-50 training on a DGX A100 (8 GPUs) with a batch size of 2,048. At a kernel-level efficiency of $\eta = 0.30$, the `DistributedModel` evaluated on a single DGX A100 node predicts a step time of ~ 69 ms, a demand rate of $\sim 29,637$ images/s. Adding the `DataModel` rules out storage I/O. With 32 CPU workers (4 per GPU) decoding ImageNet JPEGs at 1,200 images/s each, the raw decode pipeline delivers 38,400 images/s. This appears sufficient, but the `TransformationModel` accounts for the full augmentation pipeline (random crop, color jitter, normalization) at 850 images/s per worker, reducing effective throughput to 27,200 images/s, below the GPUs’ 29,637 images/s appetite. The student discovers that the binding constraint shifts from the accelerators to Wall 9 (Transformation). The GPUs have spare cycles, accelerator utilization falls to 92%, and the achieved end-to-end efficiency drops to $\eta \approx 0.28$, the same kind of gap between kernel-level and end-to-end efficiency that Anchor 1 absorbs into $\eta = 0.19$ (Section 6.1).

7.2 Instructor Use Cases

Instructors need demonstrations that run in real time, produce concrete numbers, and connect cleanly to lecture narratives. The following cases show how MLSYS·IM turns abstract concepts into live, interactive classroom demonstrations.

7.2.1 I1: Live Roofline Demo, Batch Size Sweep

An instructor demonstrates the Roofline model by sweeping batch size from 1 to 256 on an H100 for ResNet-50 inference at $\eta = 0.5$. At each batch size, the `SingleNodeModel` returns the bottleneck label, the arithmetic intensity, and the MFU:

Batch	Bottleneck	AI (FLOP/B)	MFU
1	Memory	146	0.02
8	Compute	712	0.10
32	Compute	1,220	0.25
128	Compute	1,486	0.40
256	Compute	1,541	0.45

The crossover from memory-bound to compute-bound occurs between batch 1 and batch 8, where the arithmetic intensity crosses the effective ridge point $\eta \times F_{\text{peak}} / \text{BW}_{\text{HBM}} \approx 148 \text{ FLOP/byte}$; batching amortizes the one-time weight read across samples, while per-sample activation traffic bounds the gain. MFU keeps climbing past the crossover ($0.10 \rightarrow$

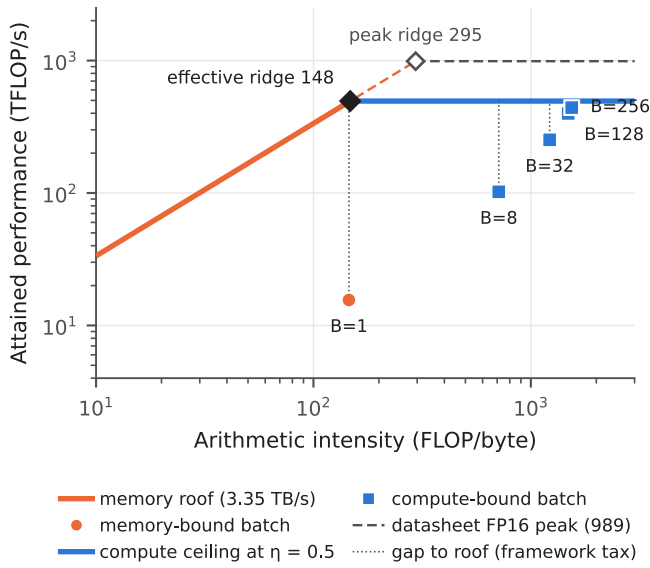


Figure 6: **Roofline Crossover: ResNet-50 Batch Size Sweep on H100.** Increasing batch size moves the operating point rightward along the Roofline, transitioning from memory-bound (purple) to compute-bound (cyan) between batch 1 and batch 8. The peak ridge point is $989/3.35 \approx 295$ FLOP/byte; at $\eta = 0.5$ the effective ridge sits at ≈ 148 FLOP/byte. MFU continues rising past the crossover as dispatch and framework overheads amortize.

0.45) as the fixed dispatch and per-layer framework taxes amortize. Figure 6 visualizes this transition on the Roofline diagram. The contrast case makes the deeper point. Repeating the sweep for a 7B-parameter LLM *decode* workload shows the arithmetic intensity saturating near 9.6 FLOP/byte, far left of the ridge, because KV-cache and activation reads grow with the batch; decode remains memory-bound at every batch size (Walls 4–5). The five-point sweep executes in about 2.41 ms, enabling real-time interaction during lecture.

7.2.2 I2: Iowa vs. Québec Carbon

An instructor poses a policy question: *does geography matter for carbon footprint?* They configure a 256-GPU cluster (`Systems.Clusters.Research_256`) running a training job for 30 days at MFU 0.42, and the `SustainabilityModel` computes emissions under two registered datacenter profiles: an illustrative high-carbon reference profile labeled Iowa ($680 \text{ gCO}_2/\text{kWh}$, PUE 1.12) and Québec (hydroelectric, $20 \text{ gCO}_2/\text{kWh}$, PUE 1.06). The configuration is identical in hardware and MFU, yet the run emits 58.4 tonnes CO_2 under the Iowa profile versus 1.6 tonnes in Québec, a $36\times$ gap, demonstrating that grid carbon intensity is a first-order systems design variable (Wall 18). Figure 7 visualizes this contrast.

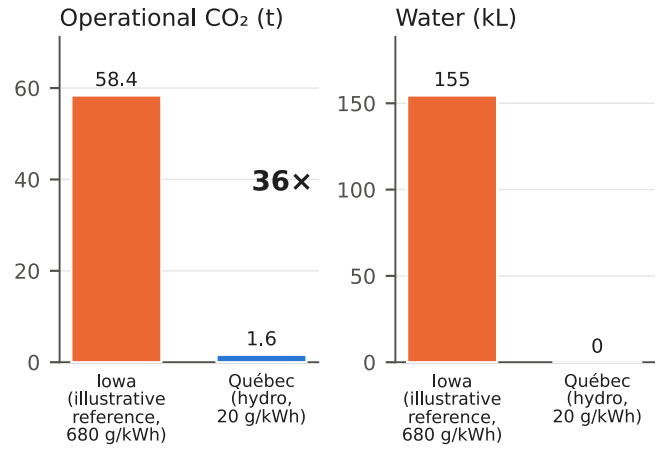


Figure 7: **Geography as a Systems Variable: Iowa vs. Québec.** An identical 256-GPU cluster running for 30 days at MFU 0.42 emits $36\times$ more CO_2 under the illustrative Iowa reference profile ($680 \text{ gCO}_2/\text{kWh}$, PUE 1.12) than in Québec (hydroelectric, $20 \text{ gCO}_2/\text{kWh}$, PUE 1.06), 58.4 vs. 1.6 tonnes. Hardware and MFU are identical; only the regional grid profile differs between the two sites.

7.3 Researcher Use Cases

Researchers need to evaluate architectural alternatives and justify procurement decisions with quantitative evidence. The following cases show how `MLSYS·IM` enables rapid what-if analysis across hardware generations and parallelism strategies.

7.3.1 R1: GPU vs. Cerebras Crossover

A researcher evaluates whether wafer-scale silicon changes the economics of large-model inference, using a user-defined 180B-parameter, 80-layer transformer (the registry has no model of this size). On H100s, the 360 GB FP16 checkpoint requires tensor parallelism across 5 GPUs, and `MLSYS·IM` reports a decode-phase ITL of $360 \text{ GB}/(5 \times 3.35 \text{ TB/s}) \approx 21.5 \text{ ms/token}$ at batch 1 (memory-bound). The `WeightStreamingModel` then models the Cerebras WSE-3 with a MemoryX injection bandwidth of 1.2 TB/s (Cerebras has not published an official MemoryX bandwidth figure, so this value is an estimate). At batch size 1 the weights must stream from MemoryX at this rate, yielding $\sim 300 \text{ ms}$ per token, far slower than the GPU cluster. Equation (14) places the optimal batch at $B^* \approx 41,666$ for $\eta = 0.4$, where injection cost would fully amortize, but at that batch the per-request KV-cache and activation state exceed the 44 GiB wafer SRAM by roughly $326\times$ and the solver marks the configuration infeasible. The largest feasible batch is 127, where the wafer delivers $\sim 423 \text{ tokens/s}$. The binding constraint therefore shifts from $\text{BW}_{\text{inject}}$ at small batches to on-wafer memory at large ones,

illustrating that the optimal architecture depends on batch size and on where request state can live.

7.3.2 R2: Hardware Procurement Audit

A researcher preparing a hardware procurement recommendation for LLM inference needs to answer: *should the next-generation cluster prioritize FLOPS or bandwidth?* The `SensitivitySolver` perturbs each hardware parameter by 10% and reports the normalized latency response, shown here for a single-GPU-resident Llama-3 8B decode workload:

```
1 import mlsysim
2 from mlsysim.solvers import SensitivitySolver
3 solver = SensitivitySolver()
4 res = solver.solve(model=mlsysim.Models.Language.
5                   Llama3_8B,
6                   hardware=mlsysim.Hardware.Cloud.
7                   A100)
8 print(res.sensitivities)
9 # {'peak_flops': 0.0, 'memory_bandwidth': -0.088,
10 #  'memory_capacity': 0.0}
11 print(f"Binding: {res.binding_constraint}")
12 # Output: Binding: memory_bandwidth
```

Listing 4: **Sensitivity Analysis.** Identifying the binding constraint via 10% hardware perturbations.

The result is unambiguous. A 10% increase in HBM bandwidth cuts decode latency by 8.8%, whereas a 10% increase in peak FLOPS yields nothing at all, because batch-1 decode sits strictly on the memory roof of Equation (3). The `SynthesisSolver` then performs the inverse solve. Given a 50 ms inter-token latency SLA for Llama-3 70B, it derives the minimum required bandwidth as $BW_{\text{req}} = |W|/T_{\text{target}} \approx 2,824 \text{ GB/s}$, $1.38\times$ the A100’s 2.04 TB/s, confirming that Llama-3 70B inference is firmly in the memory-bound regime and that hardware procurement should prioritize HBM bandwidth over peak throughput.

7.3.3 R3: End-to-End Llama-3 70B Training Audit

To illustrate how `MLSYS-IM`’s solvers compose across the taxonomy, we trace a training analysis for Llama-3 70B on the registered `Training_512_H100` fleet (64 DGX H100 nodes and 512 GPUs, NVLink intra-node and 200 Gb/s IB HDR inter-node) with a global batch of 1,024 sequences of 4,096 tokens, $TP=8$ within each node, $DP=64$, and $\eta = 0.40$.

Node (Walls 1–2). Without optimizer sharding or recomputation, the $TP=8$ training state exceeds an 80 GB H100, so the audit adds ZeRO-1 optimizer sharding across the DP group and full activation recomputation. The `DistributedModel` then evaluates one tensor-parallel replica with the Roofline core and predicts 11.70 s of compute per step, and the `TrainingMemoryModel` reports 48.6 GB of training

state per GPU, which fits the HBM. Recomputation costs a fourth forward pass, so the replica’s model FLOPs utilization is 30.0% at $\eta = 0.40$.

Algorithm (Wall 11). A 30-day run at the predicted step time processes $\sim 0.82T$ tokens, a budget of $C \approx 3.48 \times 10^{23}$ FLOPs, for which the `ScalingModel` gives a Chinchilla-optimal model size of $P^* \approx 54B$.

Fleet (Walls 14–15). Communication adds 1,338 ms of tensor-parallel AllReduce and 1,391 ms of data-parallel gradient synchronization per step, of which 209 ms is exposed after compute–communication overlap, for a step time of 13.25 s, a scaling efficiency of 88.3%, and a fleet MFU of 26.5%. The `ReliabilityModel` estimates a fleet MTBF of 67.0 hours, a Young–Daly checkpoint interval of ~ 90 minutes, about 11 failures over 30 days, and a goodput of 98.8%.

Operations (Walls 17–18). In Québec, the `EconomicsModel` projects \$351K of amortized CapEx, \$15.9K of energy, and \$53K of maintenance, a run TCO of $\sim \$0.42M$. The `SustainabilityModel` estimates 133 MWh of facility energy and 2.7 tonnes CO_2 , $36\times$ less than the 95.4 tonnes under the Iowa reference profile.

Analysis (Walls 21–22). The `SensitivitySolver` and `SynthesisSolver` operate on a single device, so a fleet-level what-if instead re-runs the chain with a changed fabric, node count, or region, which takes ~ 0.001 s per evaluation.

This trace exercises Walls 1, 2, 11, 14, 15, 17, and 18 through a single model, demonstrating how solver composition produces a system assessment from individual physics-based constraint equations.

7.3.4 R4: Automated Parallelism Search

A researcher needs to schedule a 175B-parameter model on a new 2,048-GPU cluster. Manually searching the 3D-parallelism space ($TP \times PP \times DP$) is error-prone. A split that maximizes DP might exceed the 80 GB HBM capacity, while a split that maximizes TP might saturate the NVLink interconnect. Instead of trial and error, the researcher invokes a Tier 3 Optimizer. They configure the `ParallelismOptimizer` with the workload, the cluster, a global batch of 2,048 sequences, and $\eta = 0.5$. It maximizes MFU over power-of-two factorizations with TP capped at the node size. A quick screen drops splits whose per-GPU weights, gradients, and Adam optimizer state exceed 90% of HBM (77.3 GB); the `DistributedModel` then prices each survivor with one-sequence pipeline microbatches and drops any whose training state, activations included, exceeds the 85.9 GB of an H100, leaving 13 candidates. In well under a second it returns $TP=8$, $PP=8$, $DP=32$ (MFU 39.6%), with TP matching the intra-node GPU count

so tensor-parallel traffic stays off the slower inter-node fabric. PP=8 is the shallowest pipeline whose per-GPU training state fits (55.1 GB); halving the pipeline depth would demand 98.7 GB. This demonstrates the power of the “engineering engine” to invert the analytical models into automated design-space synthesis.

8 Fallacies & Pitfalls

Following the tradition of [Hennessy et al. \(2024\)](#), we highlight common misconceptions that MLSYS·IM is designed to expose.

Fallacy: Tripling peak FLOP/s triples LLM serving speed. A student might assume that upgrading from A100 (312 TFLOP/s FP16) to H100 (989 TFLOP/s, a $3.2\times$ increase) should yield a $3.2\times$ speedup. MLSYS·IM’s `SingleNodeModel` reveals why this is false. LLM inference is memory-bandwidth-bound, and the A100-to-H100 bandwidth improvement is only about $1.6\times$ ($2.04\text{ TB/s} \rightarrow 3.35\text{ TB/s}$). For memory-bound decode, token latency scales with bandwidth, not FLOP/s; compute-bound training does see most of the FLOP/s gain. The Roofline model (Equation (3)) makes this visible by showing that the binding constraint determines which hardware parameter matters.

Fallacy: At 512 GPUs, checkpointing is a bigger tax than communication. On the 200 Gb/s IB HDR fleet of Case R3, the `DistributedModel` charges 1,338 ms of tensor-parallel and 209 ms of exposed data-parallel communication to a 13.25 s step, a 11.7% loss of scaling efficiency. The `ReliabilityModel`, with a fleet MTBF of 67.0 hours and Young–Daly checkpoints every ~ 90 minutes, charges only 1.2% of goodput to checkpoints and restarts, and that estimate is optimistic because it omits rework. At this scale communication (Wall 14) is the larger tax, and an analysis that blames checkpointing without measuring both would invest in the wrong fix.

Pitfall: Using peak bandwidth in back-of-the-envelope calculations. Vendor datasheets report peak HBM bandwidth (e.g., 3.35 TB/s for H100), and sustained bandwidth under real access patterns can be lower, so decode estimates built on peak bandwidth are optimistic. MLSYS·IM applies η only to FLOP/s; decode latency uses the registry bandwidth directly, so users who want a sustained-bandwidth estimate should derate `memory.bandwidth` explicitly (e.g., through `model_copy`).

Pitfall: Ignoring geography in carbon accounting. Two identical training runs produce vastly different environmental impact depending on grid carbon intensity. As demonstrated in Case I2, the same 256-GPU cluster emits $36\times$ more CO₂ under the Iowa reference profile

than in Québec. Students who omit Wall 18 (Sustainability) from their analysis miss a first-order systems design variable, one that increasingly affects both cost (carbon pricing) and regulatory compliance.

Fallacy: Quantization always provides a linear speedup. Reducing precision from FP16 to INT4 ($r_{\text{quant}} = 4\times$) reduces memory by $4\times$, but the compute speedup depends on whether the workload is memory-bound or compute-bound. For memory-bound LLM decode, most of the $4\times$ memory reduction becomes throughput because the bottleneck is weight reads. For compute-bound workloads, the gain depends on low-precision kernel support, so MLSYS·IM applies the registry’s per-precision FLOP/s where an entry exists and falls back to FP16 throughput, with a warning, when it does not. MLSYS·IM’s Roofline analysis makes this regime-dependent behavior explicit.

9 Discussion & Limitations

“All models are wrong, but some are useful” ([Box, 1979](#)). Users must understand the boundaries of MLSYS·IM’s analytical abstraction. We organize the limitations into modeling scope, accuracy trade-offs, and pedagogical implications, then outline future directions.

9.1 What MLSYS·IM Cannot Model

No microarchitectural effects. MLSYS·IM has no notion of L1/L2 cache hierarchies, branch prediction, kernel scheduling, or register pressure. These second-order effects are absorbed into a single scalar efficiency parameter (η , the ratio of sustained to peak FLOP/s). While η provides a serviceable approximation for back-of-the-envelope reasoning, it cannot capture workload-dependent microarchitectural behavior; a matrix multiply and a sparse attention kernel may achieve very different η on identical silicon.

No real network congestion. The communication model uses the classical $\alpha\text{-}B_{\text{link}}$ formulation (latency plus inverse-bandwidth), which assumes dedicated links. MLSYS·IM does not model adaptive routing, network contention under multi-tenant traffic, or congestion collapse, phenomena that become critical at scales beyond $\sim 10,000$ nodes, precisely the regime where ASTRA-sim 2.0 ([Won et al., 2023](#)) provides essential fidelity. The static bisection view behind Wall 10 (Locality) is optimistic under the same cross-traffic conditions. A scalar `congestion_factor` can inflate communication time, but contention dynamics are not modeled.

Coarse OS/runtime overhead. Kernel launch and framework dispatch are folded into a fixed per-device dispatch tax and a per-layer framework tax; CUDA stream scheduling, Python GIL contention, and host–

device transfer dynamics are not modeled separately, so small-batch inference predictions are only as good as those two constants.

No heterogeneous fleets. The `DistributedModel` assumes homogeneous nodes, where all accelerators in a fleet share the same compute, memory, and interconnect specifications. Production clusters increasingly mix hardware generations (e.g., A100 and H100 nodes in the same job), and fleet-level efficiency metrics such as ML Productivity Goodput (Wongpanich et al., 2025) capture this heterogeneity. Modeling heterogeneous fleets would require per-node load balancing and straggler analysis beyond the current analytical framework.

No dynamic behavior. `MLSYS·IM` models steady-state throughput. Transient effects (thermal throttling, dynamic clock boosting, memory fragmentation over long training runs, and checkpoint I/O contention with concurrent training traffic) are outside its scope. A training run that degrades over 72 hours due to thermal saturation will appear identical to one that sustains peak throughput.

Heuristic accuracy models. The accuracy degradation curves in `CompressionModel` are heuristic step functions (for quantization) and exponentials (for pruning), not architecture-specific empirical fits. Real accuracy loss depends on model architecture, calibration methodology, and quantization method (e.g., well-calibrated GPTQ INT4 can achieve <0.5% degradation, while naive round-to-nearest may lose 10%+). Users should treat these curves as directional indicators, not ground truth.

9.2 Walls Not Included

The 22-wall taxonomy is comprehensive but not exhaustive. Several constraints were considered and excluded, each for a specific reason.

Thermal throttling. Sustained power density can force throughput below peak TDP, but this is absorbed into η rather than modeled as a distinct wall.

Resource fragmentation. Scattered GPU availability across nodes prevents job scheduling even when aggregate capacity is sufficient; this is a combinatorial bin-packing problem beyond the current analytical framework.

Compiler/graph optimization. The gap between a framework’s computational graph and the executed kernel schedule affects both latency and MFU, but varies too rapidly across software versions to model analytically.

The selection criterion for inclusion was: does the constraint have a stable, published analytical formulation that remains valid across hardware generations? Constraints requiring empirical trace data or combinatorial optimization were deferred to future work.

9.3 The Accuracy–Speed Trade-off

Each omission above reflects the same trade-off: evaluation that is orders of magnitude faster, at the cost of second-order fidelity. The precedent is SPIM, which executes MIPS programs at the instruction-set level and omits pipelining, superscalar execution, and cache hierarchies, prioritizing pedagogical clarity over the full complexity of commercial processors. `MLSYS·IM` applies the same philosophy to ML systems, making quantitative reasoning accessible to students who may never operate a production cluster.

The relevant question is not “How accurate is `MLSYS·IM`?” but “Does it identify the correct binding constraint?” A first-order model that correctly determines whether a system is memory-bound, compute-bound, or network-bound provides actionable architectural insight even when its absolute latency prediction is $\pm 20\%$ from a cycle-accurate trace. The binding constraint dictates which hardware investment yields the largest return, and this ordinal ranking is far more robust than cardinal predictions. Practitioners who know that their system is memory-bandwidth-bound will invest in higher-bandwidth memory regardless of whether the predicted latency is 47 ms or 53 ms.

9.4 Future Work

We identify several directions for extending `MLSYS·IM`.

Broader empirical validation. Section 6 compares seven anchors across four taxonomy domains. Future work will extend this to newer hardware generations already present in the registry (NVIDIA B200/GB200, TPU v6) as published benchmark results appear, inference serving under load (the continuous-batching model now takes a request-length distribution, but no serving anchor validates it yet), and checkpoint overhead at scale, where published data points are becoming available from reproducibility studies.

Community hardware registry. The `Hardware` registry currently contains a curated set of hardware entries sourced to manufacturer datasheets where available, with estimates labeled as such in their provenance. We plan to open contributions from the community, with automated verification scripts that cross-check submitted specifications against known physical limits (e.g., memory bandwidth cannot exceed pin count $\times$ data rate).

Custom degradation curves. Future versions will allow users to supply empirically fitted accuracy-compression curves from their own quantization experiments, replacing the current heuristic step and exponential curves with data-grounded models.

TinyTorch integration. `MLSYS·IM` provides analytical predictions; TinyTorch (Reddi et al., 2025b),

the companion educational framework, provides implementation-based verification. Connecting the two tools creates a predict-then-verify loop. Students estimate training time and memory consumption in `MLSYS·IM`, then run the actual training in `TinyTorch` and compare. This closed loop reinforces quantitative reasoning by grounding analytical models in empirical observation.

Agentic and embodied extensions. The package already ships two extension registries, **Agents** (agentic workload profiles) and **Embodied** (robot platforms), together with the `agents` and `robotics` physics modules (Table 4). They cover trajectory reliability, test-time compute cost, and multi-agent coordination for agents, and sensor-to-actuator latency, safe stopping distance, and action-chunk cadence for robots. We deliberately keep them outside the wall taxonomy. Several of their formulas are modeling assumptions rather than published analytical results, many registry values are labeled estimates, and no resolver or validation anchor exercises them yet. Promoting candidates such as a trajectory-reliability wall or a sensor-to-actuator latency wall would require sourced formulations, dedicated resolvers, and anchors of the kind in Section 6.

Expanding Tier 3 Optimizers (Pareto Frontiers). Currently, the Tier 3 optimizers search single-dimensional objective spaces (e.g., maximizing MFU or maximizing batch size under a latency constraint); a two-objective compression Pareto sweep already ships in `CompressionModel.sweep`. Future work will extend the Tier 3 engine to support multi-objective Pareto frontiers, simultaneously optimizing across latency, total cost, carbon footprint, and accuracy. This will enable richer design-space exploration and formally expose the inherent tensions between performance and sustainability.

9.5 The Pedagogical Argument

Even when `MLSYS·IM`’s predictions miss measured values by tens of percent, as Anchors 3 and 4 do (Section 6.1), the pedagogical value lies in the reasoning process rather than the numerical output. A student who sweeps 1,000 configurations and identifies memory bandwidth as the binding constraint for LLM inference has acquired a transferable analytical skill, that of determining which resource limits performance. The framework trains students to formulate the correct quantitative questions (arithmetic intensity, ridge point location, communication-to-computation ratio) and these questions generalize to production systems even as specific hardware parameters change across generations.

10 Conclusion

Modern ML systems are no longer limited by a single resource that can be optimized in isolation. A deployment decision can hinge simultaneously on arithmetic intensity, memory capacity, communication topology, queueing delay, energy price, regional carbon intensity, and failure recovery. This coupling makes intuition brittle: a faster accelerator may not improve decode latency, a larger batch may move the bottleneck from memory to compute, and a cheaper datacenter may be the wrong choice once emissions or water constraints are included. The practical challenge is to make these interactions visible before teams commit to hardware, cloud spend, or weeks of empirical benchmarking.

`MLSYS·IM` treats ML system design as a set of executable constraints. Its purpose is not to replace profilers, trace-driven simulators, or cycle-level models; those tools remain necessary when the question is precise performance on a concrete system. Instead, `MLSYS·IM` occupies the earlier stage where engineers and students need to ask which wall binds, which parameter matters, and which design alternatives are worth measuring at all. Its standard is first-order fidelity: close enough to identify bottlenecks and rank alternatives, explicit enough to expose assumptions, and fast enough to explore the design space before measurement begins. By separating demand from supply, carrying units through its hardware, workload, and network calculations, and composing first-principles resolvers across the full stack, it turns informal back-of-the-envelope reasoning into a reproducible infrastructure model.

That artifact is also pedagogical. Paired with `TinyTorch` (Reddi et al., 2025b) and the open *Machine Learning Systems* text (Reddi et al., 2025a), `MLSYS·IM` gives learners a way to connect equations, code, and infrastructure consequences without requiring access to a GPU cluster. The goal is not merely to compute a latency or carbon number, but to teach a durable habit of systems reasoning: identify the binding constraint, check the units, perturb the design, and explain why the result changed. As ML systems continue to scale across larger models, heterogeneous fleets, and stricter economic and environmental constraints, that habit will matter as much as any single modeling tool.

Generative AI Usage Statement

In developing `MLSYS·IM`, the goal was not to formulate new physical laws or mathematical bounds, but to ground ML systems reasoning in first principles—distilling established equations and published constraints from across computer architecture, distributed systems, and machine learning

into an executable, dimensionally strict engineering framework. The package was designed specifically as the quantitative substrate for the *Machine Learning Systems* textbook series (Reddi et al., 2025a), providing an environment for first-principles architectural reasoning without requiring hardware clusters. Generative AI tools were used throughout this process to assist with cross-disciplinary literature search, locate empirical data points, and draft initial code and test scaffolding from structural blueprints. The author conceived the system architecture, structured the 22-wall taxonomy and progression, designed the software interfaces, and audited every formula, datasheet constant, and solver implementation against its primary source. All calculations are verified by automated dimensional and regression tests, and the author remains solely responsible for the design, implementation, and conclusions presented in this work.

References

- Martin Abadi, Andy Chu, Ian Goodfellow, et al. Deep learning with differential privacy. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, pages 308–318. ACM, 2016. doi: 10.1145/2976749.2978318. URL <https://doi.org/10.1145/2976749.2978318>.
- Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming Throughput-Latency tradeoff in LLM inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 117–134. USENIX Association, July 2024a. ISBN 978-1-939133-40-3. URL <https://www.usenix.org/conference/osdi24/presentation/agrawal>.
- Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Vidur: A large-scale simulation framework for LLM inference. In *Proceedings of Machine Learning and Systems (MLSys)*. mlsys.org, 2024b. URL <https://arxiv.org/abs/2405.05465>.
- Abhimanyu Bambhaniya, Ritik Raj, Geonhwa Jeong, Souvik Kundu, Sudarshan Srinivasan, Midhilesh Elavazhagan, Madhu Kumar, and Tushar Krishna. Demystifying platform requirements for diverse LLM inference use cases. *arXiv preprint arXiv:2406.01698*, 2024.
- Luiz André Barroso and Urs Hölzle. The case for energy-proportional computing. *Computer*, 40(12):33–37, 2007. ISSN 0018-9162. doi: 10.1109/mc.2007.443. URL <https://doi.org/10.1109/mc.2007.443>.
- Luiz André Barroso, Urs Hölzle, and Parthasarathy Ranganathan. *The Datacenter as a Computer*. Synthesis Lectures on Computer Architecture. Springer International Publishing, 3rd edition, 2018. ISBN 9783031006333, 9783031017612. doi: 10.1007/978-3-031-01761-2. URL <https://doi.org/10.1007/978-3-031-01761-2>.
- Nathan Binkert, Bradford Beckmann, Gabriel Black, et al. The gem5 simulator. *ACM SIGARCH Computer Architecture News*, 39(2):1–7, 2011. ISSN 0163-5964. doi: 10.1145/2024716.2024718. URL <https://doi.org/10.1145/2024716.2024718>.
- George E. P. Box. Robustness in the strategy of scientific model building. In *Robustness in Statistics*, pages 201–236. Elsevier, 1979. doi: 10.1016/B978-0-12-438150-6.50018-2. URL <https://doi.org/10.1016/B978-0-12-438150-6.50018-2>.
- Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, et al. PaLM: Scaling language modeling with pathways. *Journal of Machine Learning Research*, 24(240): 1–113, 2023.
- Benoit Courty, Victor Schmidt, et al. mlco2/codecarbon: v2.4.1. Zenodo, 2024. URL <https://doi.org/10.5281/zenodo.11171501>.
- Russ Cox, M. Frans Kaashoek, and Robert Morris. xv6: A simple, Unix-like teaching operating system. MIT PDOS, 2011. URL <https://pdos.csail.mit.edu/6.828/xv6>.
- John T. Daly. A higher order estimate of the optimum checkpoint interval for restart dumps. *Future Gener. Comput. Syst.*, 22(3):303–312, 2006. ISSN 0167-739X. doi: 10.1016/j.future.2004.11.016. URL <https://doi.org/10.1016/j.future.2004.11.016>.
- Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems 35*, volume 35, pages 16344–16359. Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2022. doi: 10.52202/068431-1189. URL <https://doi.org/10.52202/068431-1189>.
- Jeffrey Dean and Luiz André Barroso. The tail at scale. *Communications of the ACM*, 56(2):74–80, 2013. ISSN 0001-0782, 1557-7317. doi: 10.1145/2408776.2408794. URL <https://doi.org/10.1145/2408776.2408794>.

- DeepSeek-AI. Insights into DeepSeek-V3: Scaling challenges and reflections on hardware for AI architectures. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, pages 1731–1745. ACM, 2025. doi: 10.1145/3695053.3731412. URL <https://doi.org/10.1145/3695053.3731412>.
- Assaf Eisenman, Kiran Kumar Matam, Steven Ingram, Dheevatsa Mudigere, Raghuraman Krishnamoorthi, Krishnakumar Nair, Misha Smelyanskiy, and Murali Annavaram. Check-N-Run: A checkpointing system for training deep learning recommendation models. In *Proceedings of the 19th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. USENIX Association, 2022.
- Ahmad Faiz, Sotaro Kaneda, Ruhan Wang, Rita Osi, Prateek Sharma, Fan Chen, and Lei Jiang. LLMCarbon: Modeling the end-to-end carbon footprint of large language models. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*. OpenReview.net, 2024.
- William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022. URL <https://www.jmlr.org/papers/v23/21-0998.html>.
- Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *Proceedings of the 11th International Conference on Learning Representations (ICLR)*. OpenReview.net, 2023.
- Amir Gholami, Sehoon Kim, Zhen Dong, et al. *A Survey of Quantization Methods for Efficient Neural Network Inference*, pages 291–326. Chapman and Hall/CRC, 2021. ISBN 9781003162810. doi: 10.1201/9781003162810-13. URL <https://doi.org/10.1201/9781003162810-13>.
- Udit Gupta, Young Geun Kim, Sylvia Lee, Jordan Tse, Hsien-Hsin S. Lee, Gu-Yeon Wei, David Brooks, and Carole-Jean Wu. Chasing carbon: The elusive environmental footprint of computing. In *IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 854–867. IEEE, 2021. doi: 10.1109/HPCA51647.2021.00076. URL <https://doi.org/10.1109/HPCA51647.2021.00076>.
- Song Han, Huizi Mao, and William J. Dally. Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding. In *Proceedings of the 4th International Conference on Learning Representations (ICLR)*. OpenReview.net, 2016.
- Mor Harchol-Balter. *Performance Modeling and Design of Computer Systems*. Cambridge University Press, 2013. doi: 10.1017/CBO9781139226424. URL <https://doi.org/10.1017/CBO9781139226424>.
- John L. Hennessy, David A. Patterson, and Christos Kozyrakis. *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann, 7th edition, 2024. ISBN 978-0443154065.
- Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, et al. Training compute-optimal large language models. In *Advances in Neural Information Processing Systems 35*, volume 35, pages 30016–30030. Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2022. doi: 10.52202/068431-2176. URL <https://doi.org/10.52202/068431-2176>.
- Mikhail Isaev, Nic McDonald, Larry Dennison, and Richard Vuduc. Calculon: A methodology and tool for high-level co-design of systems and large language models. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–14. ACM, 2023. doi: 10.1145/3581784.3607102. URL <https://doi.org/10.1145/3581784.3607102>.
- Zhihao Jia, Matei Zaharia, and Alex Aiken. Beyond data and model parallelism for deep neural networks. In *Proceedings of Machine Learning and Systems (MLSys)*. mlsys.org, 2019. URL <https://arxiv.org/abs/1807.05358>.
- Norman P. Jouppi, George Kurian, Sheng Li, Peter Ma, Rahul Nagarajan, Lifeng Nai, Nishant Patil, Suvinay Subramanian, Andy Swing, Brian Towles, Cliff Young, Xiang Zhou, Zongwei Zhou, and David Patterson. TPU v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings. In *International Symposium on Computer Architecture (ISCA)*. ACM, 2023. doi: 10.1145/3579371.3589350. URL <https://doi.org/10.1145/3579371.3589350>.
- John Kim, William J. Dally, Steve Scott, and Dennis Abts. Technology-driven, highly-scalable dragonfly topology. In *International Symposium on Computer Architecture (ISCA)*, pages 77–88. IEEE, 2008. doi: 10.1109/ISCA.2008.19. URL <https://doi.org/10.1109/ISCA.2008.19>.
- Vijay Anand Korthikanti, Jared Casper, Sangkug Lym, Lawrence McAfee, Michael Andersch, Mohammad Shoeybi, and Bryan Catanzaro. Reducing activation recomputation in large transformer models. *Proceedings of Machine Learning and Systems*, 5, 2023. URL

- https://proceedings.mlsys.org/paper_files/paper/2023/hash/80083951326cf5b35e5100260d64ed81-Abstract-mlsys2023.html.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, et al. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 611–626. ACM, 2023. doi: 10.1145/3600006.3613165. URL <https://doi.org/10.1145/3600006.3613165>.
- Charles E. Leiserson. Fat-trees: Universal networks for hardware-efficient supercomputing. *IEEE Transactions on Computers*, C-34(10):892–901, 1985. ISSN 0018-9340. doi: 10.1109/tc.1985.6312192. URL <https://doi.org/10.1109/tc.1985.6312192>.
- Dmitry Lepikhin, Hyoungho Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. GShard: Scaling giant models with conditional computation and automatic sharding. *arXiv preprint arXiv:2006.16668*, 2020. URL <https://arxiv.org/abs/2006.16668>.
- Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*. PMLR, 2023.
- Cheng Li. llm-analysis: Latency and memory analysis of transformer models for training and inference. <https://github.com/cli99/llm-analysis>, 2023. Accessed: 2025-01-15.
- Mingyu Liang, Hiwot Tadesse Kassa, Wenyin Fu, Brian Coutinho, Louis Feng, and Christina Delimitrou. Lumos: Efficient performance modeling and estimation for large-scale LLM training. In *Proceedings of Machine Learning and Systems (MLSys)*. mlsys.org, 2025.
- Sean Lie. Cerebras architecture deep dive: First look inside the HW/SW co-design for deep learning. In *IEEE Hot Chips 34 Symposium*. IEEE, 2022.
- Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration. *GetMobile: Mobile Computing and Communications*, 28(4):12–17, 2024. ISSN 2375-0529, 2375-0537. doi: 10.1145/3714983.3714987. URL <https://doi.org/10.1145/3714983.3714987>.
- Llama Team, AI Meta. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- Jayashree Mohan, Amar Phanishayee, Ashish Raniwala, and Vijay Chidambaram. Analyzing and mitigating data stalls in DNN training. *Proc. VLDB Endow.*, 14(5): 771–784, 2021. ISSN 2150-8097. doi: 10.14778/3446095.3446100. URL <https://doi.org/10.14778/3446095.3446100>.
- Derek G. Murray, Jiri Simsa, Ana Klimovic, and Ihor Indyk. tf.data: A machine learning data processing framework. *Proceedings of the VLDB Endowment*, 14 (12):2945–2958, 2021.
- Deepak Narayanan, Mohammad Shoeybi, Jared Casper, et al. Efficient large-scale language model training on GPU clusters using Megatron-LM. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–15. ACM, 2021. doi: 10.1145/3458817.3476209. URL <https://doi.org/10.1145/3458817.3476209>.
- NVIDIA Corporation. ResNet-50 v1.5 for TensorFlow: Training performance on NVIDIA DGX A100. NVIDIA DeepLearningExamples repository, 2020. URL <https://github.com/NVIDIA/DeepLearningExamples/tree/master/TensorFlow/Classification/ConvNets/resnet50v1.5>. TensorFlow 20.06-tf1-py3 NGC container, 8x A100 40GB, batch size 256 per GPU, mixed precision with XLA. Accessed: 2026-09-15.
- NVIDIA Corporation. NVIDIA H100 tensor core GPU: Product specifications. <https://www.nvidia.com/en-us/data-center/h100/>, 2023. Accessed: 2026-09-15.
- Angshuman Parashar, Priyanka Raina, Yakun Sophia Shao, Yu-Hsin Chen, Joel Emer, et al. Timeloop: A systematic approach to DNN accelerator evaluation. In *2019 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 304–315. IEEE, 2019. doi: 10.1109/ispass.2019.00042. URL <https://doi.org/10.1109/ispass.2019.00042>.
- Pitch Patarasuk and Xin Yuan. Bandwidth optimal all-reduce algorithms for clusters of workstations. *Journal of Parallel and Distributed Computing*, 69(2):117–124, 2009. doi: 10.1016/j.jpdc.2008.09.002. URL <https://doi.org/10.1016/j.jpdc.2008.09.002>.
- Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative LLM inference

- using phase splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, pages 118–132. IEEE, 2024. doi: 10.1109/isca59077.2024.00019. URL <https://doi.org/10.1109/isca59077.2024.00019>.
- David Patterson, Joseph Gonzalez, Quoc Le, et al. Carbon emissions and large neural network training. *arXiv preprint arXiv:2104.10350*, 2021.
- David A. Patterson and John L. Hennessy. *Computer Organization and Design: The Hardware/Software Interface*. Morgan Kaufmann, 5th edition, 2014. ISBN 978-0124077263.
- Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, et al. Efficiently scaling transformer inference. In *Proceedings of Machine Learning and Systems (MLSys)*, volume 5. mlsys.org, 2023. URL https://proceedings.mlsys.org/paper_files/paper/2023/hash/c4be71ab8d24cdfb45e3d06dbfca2780-Abstract-mlsys2023.html.
- Hang Qi, Evan R. Sparks, and Ameet Talwalkar. PALEO: A performance model for deep neural networks. In *Proceedings of the 5th International Conference on Learning Representations (ICLR)*. OpenReview.net, 2017.
- Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. ZeRO: Memory optimizations toward training trillion parameter models. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–16. IEEE, 2020. doi: 10.1109/sc41405.2020.00024. URL <https://doi.org/10.1109/sc41405.2020.00024>.
- Vijay Janapa Reddi et al. *Machine Learning Systems: Principles and Practices of Engineering Artificially Intelligent Systems*. Harvard University, 2025a. URL <https://mlsysbook.ai>.
- Vijay Janapa Reddi et al. TinyTorch: A progressive educational framework for machine learning systems. Harvard University, 2025b. URL <https://mlsysbook.ai/tinytorch>.
- Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, et al. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *Proceedings of the 5th International Conference on Learning Representations (ICLR)*. OpenReview.net, 2017.
- Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-LM: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.
- Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. In *Proceedings of the 13th International Conference on Learning Representations (ICLR)*. OpenReview.net, 2025.
- Arthur G. Stephenson, Lia S. LaPiana, Daniel R. Mulville, Peter J. Rutledge, Frank H. Bauer, David Folta, Greg A. Dukeman, Robert Sackheim, and Peter Norvig. Mars Climate Orbiter Mishap Investigation Board Phase I Report. Technical report, National Aeronautics and Space Administration, 1999.
- Rich Sutton. The bitter lesson. <http://www.incompleteideas.net/IncIdeas/BitterLesson.html>, 2019. Accessed: 2024-06-15.
- Andrew S. Tanenbaum and Albert S. Woodhull. *Operating Systems: Design and Implementation*. Prentice Hall, 3rd edition, 2006.
- Xizheng Wang et al. SimAI: Unifying architecture design and performance tuning for large-scale large language model training with scalability and precision. In *Proceedings of the 22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. USENIX Association, 2025.
- Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009. ISSN 0001-0782, 1557-7317. doi: 10.1145/1498765.1498785. URL <https://doi.org/10.1145/1498765.1498785>.
- William Won, Taekyung Heo, Saeed Rashidi, Srinivas Sridharan, Sudarshan Srinivasan, and Tushar Krishna. ASTRA-sim2.0: Modeling hierarchical networks and disaggregated systems for large-model training at scale. In *2023 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 283–294. IEEE, 2023. doi: 10.1109/ispass57527.2023.00035. URL <https://doi.org/10.1109/ispass57527.2023.00035>.
- Arissa Wongpanich, Tayo Oguntebi, Jose Baiocchi Paredes, Yu Emma Wang, Phitchaya Mangpo Phothilimthana, Ritwika Mitra, Zongwei Zhou, Naveen Kumar, and Vijay Janapa Reddi. Machine learning fleet efficiency: Analyzing and optimizing large-scale Google TPU systems with ML productivity goodput, 2025. arXiv:2502.06982.
- Yannan Nellie Wu, Joel S. Emer, and Vivienne Sze. Accelerergy: An architecture-level energy estimation methodology for accelerator designs. In *2019 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*,

- pages 1–8. IEEE, 2019. doi: 10.1109/iccad45719.2019.8942149. URL <https://doi.org/10.1109/iccad45719.2019.8942149>.
- Wm. A. Wulf and Sally A. McKee. Hitting the memory wall. *ACM SIGARCH Computer Architecture News*, 23(1):20–24, 1995. doi: 10.1145/216585.216588. URL <https://doi.org/10.1145/216585.216588>.
- John W. Young. A first order approximation to the optimum checkpoint interval. *Communications of the ACM*, 17(9):530–531, 1974. ISSN 0001-0782, 1557-7317. doi: 10.1145/361147.361115. URL <https://doi.org/10.1145/361147.361115>.
- Geoffrey X. Yu, Yubo Gao, Pavel Golikov, and Gennady Pekhimenko. Habitat: A runtime-based computational performance predictor for deep neural network training. In *Proceedings of the 2021 USENIX Annual Technical Conference (ATC)*. USENIX Association, 2021.
- Zhihang Yuan, Yuzhang Shang, Yang Zhou, et al. LLM inference unveiled: Survey and roofline model insights. *arXiv preprint arXiv:2402.16363*, 2024.
- Hengrui Zhang, August Ning, Rohan Baskar Prabhakar, and David Wentzlaff. LLMCompass: Enabling efficient hardware design for large language model inference. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, pages 1080–1096. IEEE, 2024. doi: 10.1109/isca59077.2024.00082. URL <https://doi.org/10.1109/isca59077.2024.00082>.
- Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, et al. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems 37*, pages 62557–62583. Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2024. doi: 10.52202/079017-2000. URL <https://doi.org/10.52202/079017-2000>.
- Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 193–210. USENIX Association, July 2024. ISBN 978-1-939133-40-3. URL <https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin>.

Appendix A MLSysBook Integration (LEGO Cells)

MLSYS·IM serves as the executable backbone for *Machine Learning Systems*, a four-volume series (<https://mlsysbook.ai/vol1>, <https://mlsysbook.ai/vol2>, <https://mlsysbook.ai/vol3>, and <https://mlsysbook.ai/vol4>) (Reddi et al., 2025a). To integrate quantitative analysis into the prose without disrupting readability, the textbook introduces the concept of **LEGO Cells**. LEGO is an acronym for the four structural phases of each cell: **Load** (importing registries and constraints), **Execute** (invoking the physics solvers), **Guard** (asserting dimensional and logical invariants), and **Output** (formatting physical quantities for inline prose). A LEGO Cell is a standalone, self-contained Python snippet that answers a specific design question posed in the text. Isolating the simulation logic into these executable blocks allows readers to read the code to understand the mechanics, execute it to verify the results, and modify the parameters to explore alternative scenarios.

Below, we provide three illustrative cells written in the LEGO style used throughout the textbook. They are simplified for this paper and are not verbatim excerpts; the cells in the books wrap the same four phases in a class with a documented header.

A.1 Volume 1 Style: Hardware Constraints

In Volume 1, LEGO Cells are heavily used to explain hardware constraints. For example, when introducing the Roofline model, a LEGO-style cell shows that Llama-3 8B decode on an H100 remains memory-bound even as batch size grows:

```
1 from mlsysim.engine.engine import Engine
2 from mlsysim.hardware.registry import Hardware
3 from mlsysim.models.registry import Models
4
5 model = Models.Language.Llama3_8B
6 hw = Hardware.Cloud.H100
7
8 for b in [1, 16, 128, 512]:
9     perf = Engine.solve(model, hw, batch_size=b, precision="fp16")
10    print(f"Batch {b}: {perf.bottleneck}-bound, MFU={perf.mfu:.3f}")
```

Listing 5: **Illustrative LEGO-Style Cell: Roofline Analysis.** Sweeping batch size with the Engine shows Llama-3 8B decode remaining memory-bound at every batch size.

A.2 Volume 2 Style: Distributed Training

In Volume 2, the focus shifts to distributed systems and fleet-level economics. LEGO-style cells here compose multiple solvers to estimate the time and cost of training frontier models across thousands of GPUs:

```
1 from mlsysim.solvers import DistributedModel, SustainabilityModel
2 from mlsysim.systems.registry import Systems
3 from mlsysim.models.registry import Models
4 from mlsysim.infrastructure.registry import Infrastructure
5
6 fleet = Systems.Clusters.Frontier_8K
7 model = Models.Language.Llama3_70B
8
9 perf = DistributedModel().solve(
10     model, fleet, batch_size=4096, seq_len=4096, tp_size=8, pp_size=8
11 )
12 training_days = (perf.step_latency_total * 1e12 / (4096 * 4096)).to("day")
13
14 sust = SustainabilityModel().solve(
15     fleet, duration_days=training_days.magnitude,
16     datacenter=Infrastructure.Datacenters.Iowa_Reference, mfu=0.45
17 )
18 print(f"Training Time: {training_days.magnitude:.0f} days")
19 print(f"Carbon: {sust.carbon_footprint_kg / 1000:.0f} tonnes")
```

Listing 6: **Illustrative LEGO-Style Cell: Training Economics.** Composing solvers to estimate the duration and carbon footprint of a large-scale training run.

A.3 Volume 1 Style: Model Serving

Another common pattern in Volume 1 evaluates inference systems. Here, a LEGO-style cell uses the `ServingModel` to calculate the inter-token latency (ITL) for Llama-3 8B on an NVIDIA A100 GPU:

```
1 from mlsysim.solvers import ServingModel
2 from mlsysim.models.registry import Models
3 from mlsysim.hardware.registry import Hardware
4
5 # 1. LOAD (Constants)
6 model = Models.Language.Llama3_8B
7 hw = Hardware.Cloud.A100
8
9 # 2. EXECUTE (The Compute)
10 perf = ServingModel().solve(
11     model, hw, seq_len=1256, batch_size=1, precision="int4", efficiency=1.0
12 )
13
14 # 3. GUARD (Invariants)
15 assert perf.feasible, "configuration must fit in memory"
16
17 # 4. OUTPUT (Formatting)
18 print(f"Inter-Token Latency (ITL): {perf.itl.to('ms'):.2f}")
```

Listing 7: **Illustrative LEGO-Style Cell: Model Serving.** The `ServingModel` estimates the theoretical inter-token latency for an autoregressive generation step. The cell shows the four **LOAD**, **EXECUTE**, **GUARD**, and **OUTPUT** phases of the LEGO pattern.

Appendix B Registry Snapshot

The MLSys Zoo comprises eight core registries spanning the full systems stack, plus the **Agents** and **Embodied** extension registries and the `Literature` and `ReferenceStats` auxiliary surfaces. This appendix is a representative snapshot of the current local registry surface: enough to show the kinds of systems `MLSYS · IM` can compose, without treating the paper as the authoritative inventory. The typed Python and YAML registry source remains the source of truth for the complete, up-to-date entries.

B.1 Hardware Registry

Table 7 summarizes the current accelerator coverage by deployment tier. Each entry is a fully typed `HardwareNode` with `pint`-dimensioned fields. Users add custom entries by instantiating the same types (Listing 2), and custom entries participate in the same dimensional pipeline as vetted ones. For example, a research team evaluating a pre-release accelerator can define a speculative entry with projected specifications and immediately sweep it against existing workloads (Listing 8).

```
1 from mlsysim.hardware.types import HardwareNode, ComputeCore, MemoryHierarchy
2 from mlsysim.core.units import TFLOPs, GB, TB, second, watt
3
4 custom_accelerator = HardwareNode(
5     name="Speculative GPU 2027", release_year=2027,
6     compute=ComputeCore(peak_flops=1200 * TFLOPs / second,
7         precision_flops={"fp16": 1200 * TFLOPs / second, "fp8": 2400 * TFLOPs / second}),
8     memory=MemoryHierarchy(capacity=144 * GB, bandwidth=5.0 * TB / second),
9     tdp=800 * watt,
10 )
```

Listing 8: **Extending the Registry.** Defining a speculative hardware entry using the typed `HardwareNode` schema (Table 9) for immediate solver evaluation.

Table 7: **Hardware Registry Snapshot.** All 45 hardware entries in the registry, organized by deployment tier and sorted by peak throughput. Peak FLOP/s is FP16 Tensor Core (or equivalent) throughput; BW is primary-memory bandwidth (HBM, LPDDR, or on-die SRAM as appropriate); memory is binary capacity in GiB.

Tier	Accelerator	Peak (TFLOP/s)	Mem (GiB)	BW (TB/s)	TDP (W)	Interconnect
<i>Cloud</i>						
	Intel SGX Enclave (Reference)	1	0.125	0.01	—	—
	Reference Desktop CPU	1	64	0.05	150	—
	Google TPU v2	45	16	0.7	200	—
	NVIDIA T4	65	16	0.32	70	—
	Google TPU v1	92	8	0.034	75	—
	Google TPU v3	105	32	0.9	250	—
	NVIDIA V100 PCIe	112	32	0.9	250	PCIe Gen3 x16
	NVIDIA V100	125	32	0.9	300	NVLink 2.0 (300 GB/s)
	Groq LPU	188	0.225	80	215	—
	Graphcore Colossus Mk2 GC200 IPU	250	0.879	45	—	—
	Google TPU v4	275	32	1.2	200	ICI (300 GB/s)
	NVIDIA A100	312	80	2.04	400	NVLink 3.0 (600 GB/s)
	AWS Trainium 2	380	96	2.4	500	—
	AMD MI250X	383	128	3.2	500	—
	Intel Gaudi 2	432	96	2.45	600	RoCE (24x100GbE) (300 GB/s)
	Google TPU v5p	459	95	2.76	300	ICI (1,200 GB/s)
	Google TPU v6e (Trillium)	918	32	1.64	300	—
	NVIDIA H100	989	80	3.35	700	NVLink 4.0 (900 GB/s)
	NVIDIA H200	989	131	4.8	700	NVLink 4.0 (900 GB/s)
	AMD MI300X	1,307	192	5.3	750	—
	Intel Gaudi 3	1,835	128	3.7	900	—
	Untether AI speedAI240	2,000	0.232	1,000	—	—
	NVIDIA B200	2,250	180	8	1,000	NVLink 5.0 (1,800 GB/s)
	d-Matrix Corsair	2,400	2	150	600	—
	Cerebras CS-3 (WSE-3)	125,000	44	21,000	23,000	SwarmX / MemoryX
	NVIDIA GB200 NVL72	162,000	13,824	576	120,000	NVLink Switch (Bisection)
<i>Workstation</i>						
	MacBook Pro (M3 Max)	14.2	128	0.4	100	—
	NVIDIA DGX Spark (GB10)	125	128	0.273	200	—
<i>Edge</i>						
	Edge Server	1	128	0.1	300	—
	Intel NUC + Movidius	1	16	0.025	15	—
	Google Coral Edge TPU	4	1	0.008	2	—
	RoboTaxi Reference Compute (NVIDIA DRIVE AGX Orin class)	5.2	32	0.2	60	Automotive sensor and Ethernet I/O
	NVIDIA Jetson Orin Nano	10	8	0.068	15	—
	NVIDIA Jetson Orin NX	25	16	0.102	25	—
	NVIDIA Jetson AGX Orin	275	64	0.204	60	—
	NVIDIA DRIVE Thor	1,000	64	0.546	100	Automotive 100G Ethernet and PCIe Gen5
<i>Mobile</i>						
	Google Pixel 8 (Tensor G3)	15	8	0.06	5	—
	Apple M2 Neural Engine	15.8	16	0.1	20	—
	iPhone 15 Pro (A17 Pro)	35	8	0.0512	5	—
	Snapdragon 8 Gen 3	45	12	0.077	5	—
<i>Tiny</i>						
	Nordic nRF52840 (Cortex-M4F)	0.000064	0.000977	0.000064	0.015	—
	Oura Ring 4 (wearable reference profile)	0.0001	0.00195	0.00008	0.02	—
	Himax WE-I Plus	0.0002	0.00195	0.0001	0.005	—
	ESP32-S3 (AI)	0.0005	0.00391	0.00008	0.4	—
	Lockstep safety MCU (reference class)	0.0008	0.00391	0.0016	1	—

Notes: Cerebras CS-3 BW is on-wafer SRAM bandwidth; the weight-streaming serving case study (Section 7) instead uses the 1.2 TB/s MemoryX injection bandwidth seen by the host. NVIDIA GB200 NVL72 is a rack-scale system entry (72 Blackwell GPUs behind one NVLink switch domain) rather than a single accelerator.

Table 8: **Infrastructure and Systems Registry Snapshot.** *Top:* Grid profiles for sustainability analysis (CI = carbon intensity in gCO₂/kWh; PUE = power usage effectiveness; WUE = water usage effectiveness in L/kWh). *Bottom:* Registered compute nodes, network fabrics, and fleet configurations for distributed analysis.

Grid Profiles (12 registered)				
Profile	CI	PUE	WUE	Source
Norway (Hydro)	10	1.06	0.0	Hydro
Quebec (Hydro)	20	1.06	0.0	Hydro
Iceland (Geothermal)	28	1.06	0.0	Geothermal
France (Nuclear)	50	1.12	1.8	Nuclear
EU Average	270	1.12	1.8	Mixed
Germany (Coal+Wind)	385	1.12	1.8	Mixed
Texas (ERCOT)	400	1.12	1.8	Mixed
US Average	429	1.12	1.8	Mixed
China Average	555	1.40	1.8	Coal
Iowa (Coal/Gas Reference)	680	1.12	1.8	Coal/gas
India Average	720	1.58	1.8	Coal
Poland (Coal)	820	1.58	1.8	Coal
Nodes, Fabrics, and Fleets				
Compute Nodes (6 registered)				
DGX A100	8× NVIDIA A100, NVLink 3.0, 300 GB/s per direction			
DGX B200	8× NVIDIA B200, NVLink 5.0, 900 GB/s per direction			
DGX H100	8× NVIDIA H100, NVLink 4.0, 450 GB/s per direction			
HGX H100 (Dual AMD EPYC 9654)	8× NVIDIA H100, NVLink 4.0, 450 GB/s per direction			
Kempner H100 4-GPU Server	4× NVIDIA H100, NVLink 4.0, 450 GB/s per direction			
MacBook Pro M3 Max Workstation	1× MacBook Pro (M3 Max), intra-node link, 400 GB/s per direction			
Network Fabrics (10 registered)				
10GbE	10 Gb/s, 50 μs			
100GbE	100 Gb/s, 50 μs			
100GbE RoCE	100 Gb/s, 10 μs			
IB HDR	200 Gb/s, 7 μs			
400GbE	400 Gb/s, 50 μs			
IB NDR	400 Gb/s, 5 μs			
800GbE	800 Gb/s, 50 μs			
IB XDR	800 Gb/s, 5 μs			
1.6TbE	1,600 Gb/s, 50 μs			
IB GXDR	1,600 Gb/s, 5 μs			
Fleet Configurations (11 registered)				
Lab Cluster (64 H100 GPUs)	8× DGX H100, IB HDR			
Research Cluster (256 GPUs)	32× DGX H100, 100GbE			
Kempner AI Cluster H100 Partition (384 H100 GPUs)	96× Kempner H100 4-GPU Server, IB NDR			
Training Cluster (512 H100 GPUs)	64× DGX H100, IB HDR			
Training Cluster (1024 A100 GPUs)	128× DGX A100, IB HDR			
Training Cluster (1024 GPUs)	128× DGX H100, IB HDR			
Production Cluster (2048 GPUs)	256× DGX H100, IB HDR			
Frontier Cluster (8192 GPUs)	1,024× DGX H100, IB NDR			
Training Cluster (10000 GPUs)	1,250× DGX H100, IB NDR			
Reference H100 Cluster (25000 GPUs)	3,125× DGX H100, IB NDR			
Mega Cluster (100000 GPUs)	12,500× DGX H100, IB NDR			

B.2 Infrastructure and Systems Registries

The Hardware registry (Table 7) captures individual accelerator capabilities, but real deployments compose accelerators into nodes, connect nodes via network fabrics, and situate clusters in datacenters whose geography determines energy cost, carbon intensity, and cooling. The Infrastructure and Systems registries complete this picture: Infrastructure encodes *where* the fleet runs, and Systems encodes *how* accelerators form distributed configurations. Together the three registries let a single resolver chain evaluate end-to-end metrics from per-device roofline through fleet-level TCO and carbon footprint.

The **Infrastructure** registry encodes regional environmental context for sustainability analysis, plus datacenter profiles, rack power envelopes, cooling, cloud pricing, and capacity lead times. Carbon intensity varies by roughly 82× across the 12 registered grid profiles, making geography a first-order design variable (Case I2, Section 7). The **Systems** registry defines compute nodes, network fabrics, fleet configurations, TPU pods, switch fabrics, and storage

Table 9: **Core Type Schema.** Pydantic BaseModel types that compose registry entries. The hierarchy is deliberately shallow: HardwareNode aggregates ComputeCore and MemoryHierarchy as direct fields, not through deep inheritance. All physical fields are pint Quantity objects. Users extend any registry by instantiating these types directly (Listing 2).

Type	Key Fields
ComputeCore	peak_flops (FLOP/s), precision_flops (dict: tf32/fp8/int8/fp32 → FLOP/s; FP16 is the base peak_flops), sm_count
MemoryHierarchy	capacity (bytes), bandwidth (bytes/s); optional SRAM, flash, and L2 tiers
IOInterconnect	bandwidth (bytes/s), latency (seconds), name
HardwareNode	compute, memory, storage, interconnect, nvlink, tdp (W), unit_cost (USD), dispatch_tax (s), metadata.provenance
ComputationGraph	total_ops (FLOPs), weight_bytes (bytes), arithmetic_intensity (FLOP/byte), parameter_count, layers
TransformerWorkload	name, architecture, parameters, layers, hidden_dim, heads, kv_heads
GridProfile	carbon_intensity_g_kwh, pue, wue (L/kWh), renewable_pct
NetworkFabric	topology, bandwidth (bytes/s), latency (s), oversubscription_ratio
Fleet	node, count, fabric (NetworkFabric), region/datacenter, mtbf_hours
SystemEvaluation	feasibility, performance, macro (three-level scorecard)

and checkpoint profiles (NVMe, SATA, HDD, object stores, parallel file systems) used by DistributedModel, ReliabilityModel, and DataModel. Table 8 presents representative entries from both registries.

B.3 Type System and Physics API

To support the separation of demand and supply, MLSYS·IM relies on a shallow but rigorously typed hierarchy of Pydantic models (Table 9). These types act as the contract between the declarative registries and the analytical solvers. Because all physical quantities (bandwidth, latency, power) are dimensioned using the pint library, users can extend any registry by instantiating these types directly without risking unit mismatch errors.

Beneath the resolvers lies the Physics Engine, which contains the dimensioned mathematical identities that govern ML systems. Table 10 catalogs the public API of these physics modules. Each function is stateless, accepting dimensioned quantities (or raw floats that are immediately coerced to units) and returning dimensioned bounds. These functions implement the core logic for the Iron Law of training, the Roofline model, and network communication topologies.

Table 10: **Physics Module Public API.** Selected function signatures organized by domain module. Every function accepts `pint Quantity` arguments (or raw floats coerced via `_ensure_unit()`) and returns dimensioned quantities. Import from `mlsysim.physics` (one module per domain).

Module	Function	Signature (key parameters → return)
performance	<code>dTime</code>	<code>(total_ops, n_devices, peak_flops, η) → seconds</code>
	<code>calc_bottleneck</code>	<code>(ops, model_bytes, device_flops, device_bw) → {time, regime}</code>
	<code>calc_amdahls_speedup</code>	<code>(parallel_frac, speedup_factor) → overall speedup</code>
	<code>calc_pipeline_bubble</code>	<code>(n_stages, n_microbatches, v_stages) → bubble fraction</code>
	<code>calc_effective_flops</code>	<code>(peak_flops, mfu, scaling_eff, goodput_ratio) → sustained FLOP/s</code>
memory	<code>calc_training_time_days</code>	<code>(total_ops, n_devices, peak_flops, η) → days</code>
	<code>model_memory</code>	<code>(params, bytes_per_param, unit) → memory</code>
	<code>calc_activation_memory</code>	<code>(n_layers, seq_len, batch_size, hidden_dim, n_heads, precision_bytes, strategy) → bytes</code>
	<code>calc_kv_cache_size</code>	<code>(layers, heads, dim, seq, batch, prec) → bytes</code>
	<code>calc_paged_kv_cache_size</code>	<code>(same + page_size) → (bytes, last-page waste %)</code>
serving	<code>calc_checkpoint_size</code>	<code>(n_params, bytes_per_param) → bytes</code>
	<code>calc_queue_latency_mmc</code>	<code>(arrival_rate, service_rate, n_servers) → wait time</code>
communication	<code>calc_ring_allreduce_time</code>	<code>(msg_bytes, n_gpus, bw, latency) → seconds</code>
	<code>calc_tree_allreduce_time</code>	<code>(msg_bytes, n_gpus, bw, latency) → seconds</code>
	<code>calc_all_to_all_time</code>	<code>(msg_bytes, n_gpus, bw, latency) → seconds</code>
	<code>calc_hierarchical_allreduce_time</code>	<code>(message_bytes, n_nodes, gpus_per_node, intra_node_bw, inter_node_bw, intra_node_lat, inter_node_lat) → seconds</code>
reliability	<code>calc_young_daly_interval</code>	<code>(ckpt_cost, mtbf) → optimal interval</code>
	<code>calc_mtbtf_cluster</code>	<code>(component_mtbtf, n, correlation) → cluster MTBF</code>
	<code>calc_mtbtf_node</code>	<code>(GPU, NIC, PSU, and other component MTBFs with counts) → node MTBF</code>
	<code>calc_availability_stacked</code>	<code>(single_avail, n_replicas) → stacked availability</code>
transformer	<code>calc_failure_probability</code>	<code>(mtbf, duration) → probability</code>
	<code>calc_transformer_training_flops</code>	<code>(n_params, n_tokens) → total FLOPs</code>
economics	<code>calc_transformer_decode_flops</code>	<code>(n_params, n_tokens) → decode FLOPs</code>
	<code>calc_fleet_tco</code>	<code>(unit_cost, power, quantity, years, kwh_price) → TCO</code>
networking	<code>calc_monthly_egress_cost</code>	<code>(bytes_per_sec, cost_per_gb) → monthly cost</code>
	<code>calc_network_latency_ms</code>	<code>(distance_km) → latency</code>
statistics	<code>calc_population_stability_index</code>	<code>(expected, actual) → PSI</code>
	<code>calc_two_proportion_sample_size</code>	<code>(baseline, lift, z_α, z_β) → n</code>
	<code>calc_constraint_propagation_factor</code>	<code>(from, to, base) → factor</code>
quantities	<code>transfer_time / compute_time</code>	<code>(payload, bandwidth) / (work, throughput) → time</code>
	<code>energy_from_power / carbon_from_energy</code>	<code>(power, duration) / (energy, CI) → energy / mass</code>
	<code>memory_from_params / token_throughput</code>	<code>(params, bytes/param) / (tokens, duration) → bytes / rate</code>

B.4 Code Examples Cookbook

To demonstrate how the theoretical 3-tier architecture from Section 5 is exposed in Python, we provide eight complete, runnable examples. Each example is written as a small engineering task: the prose names the question, the listing shows which registry facts are selected, and the returned object exposes the bound, bottleneck, or scorecard used to

make a decision. The point is not to hide the model behind an API call, but to show how registry-backed composition turns a systems question into a repeatable calculation.

B.4.1 Scenario A: Rapid Parametric Sweeps (Tier 1)

Scenario: You want to know how much decode throughput continuous batching can buy on an H100 before KV-cache and activation traffic saturate the memory system. The code selects a workload from `Models.Language` and a supply point from `Hardware.Cloud`, then calls `Engine.solve()` once per batch size. Each returned performance object reports throughput, MFU, and the binding Roofline regime, so the sweep shows not only that throughput saturates but why it remains memory-bound.

```
1 from mlsysim.engine.engine import Engine
2 from mlsysim.hardware.registry import Hardware
3 from mlsysim.models.registry import Models
4 import pandas as pd
5
6 # Sweep Llama 3 8B across multiple batch sizes on an H100 GPU
7 model = Models.Language.Llama3_8B
8 hardware = Hardware.Cloud.H100
9
10 results = []
11 for batch_size in [1, 8, 32, 128, 256]:
12     # Engine.solve() wraps the Tier 1 SingleNodeModel
13     perf = Engine.solve(model, hardware, batch_size=batch_size, precision="fp16")
14
15     results.append({
16         "Batch Size": batch_size,
17         "Throughput (tok/s)": round(perf.throughput.magnitude),
18         "Bottleneck": perf.bottleneck,
19         "MFU": round(perf.mfu, 3)
20     })
21
22 df = pd.DataFrame(results)
23 print(df)
24 #   Batch Size Throughput (tok/s) Bottleneck    MFU
25 # 0    1    178    Memory    0.003
26 # 1    8    893    Memory    0.014
27 # 2   32   1,564    Memory    0.025
28 # 3   128   1,925    Memory    0.031
29 # 4   256   2,002    Memory    0.033
```

Listing 9: **Programmatic Sweeps (Tier 1).** Using the Engine to sweep a workload across a hardware constraint space (batch size). Throughput rises from 178 to 2,002 tokens/s and then saturates: per-request memory traffic grows with the batch, so decode stays memory-bound at every point. Because execution takes <1 ms per point, sweeping hundreds of configurations is trivial.

B.4.2 Scenario B: Cross-Domain Carbon Accounting (Tier 1 Composition)

Scenario: You have been tasked with estimating the carbon footprint of training a 70B parameter model. The listing first uses `DistributedModel` to compute the step latency for a registry-backed 256-GPU fleet, then converts a one-trillion-token target into a training duration. That duration becomes the input to `SustainabilityModel`, where the Iowa datacenter profile supplies grid intensity and facility overhead; the composition works because solver outputs are dimensioned quantities rather than untyped floats.

```
1 from mlsysim.solvers import DistributedModel, SustainabilityModel
2 from mlsysim.systems.registry import Systems
3 from mlsysim.infrastructure.registry import Infrastructure
4 from mlsysim.models.registry import Models
5
6 fleet = Systems.Clusters.Research_256 # 32x DGX H100 nodes
7
8 # 1. Performance Domain: how long does one training step take?
9 perf = DistributedModel().solve(
10     model=Models.Language.Llama3_70B, fleet=fleet,
```

```

11     batch_size=1024, seq_len=2048, tp_size=8, pp_size=1,
12     activation_recomputation=True,
13 )
14
15 # 2. Train for 1T tokens: convert step latency into a duration
16 steps = 1e12 / (1024 * 2048)
17 training_days = (perf.step_latency_total * steps).to("day").magnitude
18
19 # 3. Sustainability Domain: what is the carbon impact?
20 sust = SustainabilityModel().solve(
21     fleet=fleet, duration_days=training_days,
22     datacenter=Infrastructure.Datacenters.Iowa_Reference, # high-carbon grid
23     mfu=0.42,
24 )
25 print(f"Training Duration: {training_days:.0f} days")
26 # Training Duration: 74 days
27 print(f"Total Carbon: {sust.carbon_footprint_kg / 1000:.1f} tonnes CO2e")
28 # Total Carbon: 144.3 tonnes CO2e

```

Listing 10: Resolver Composition (Tier 1). Output quantities from one solver (the computed step latency from the DistributedModel) feed directly into subsequent solvers (the SustainabilityModel), enabling end-to-end full-stack analysis without manual unit conversion.

B.4.3 Scenario C: Uncovering the Data Wall (Tier 1)

Scenario: Your GPU utilization is mysteriously low during a computer vision training job. The code splits the diagnosis into two checks: DataModel compares the required image stream against the node’s I/O path, while TransformationModel compares CPU preprocessing throughput against accelerator step time. Reading the two results together distinguishes a storage or transfer bottleneck from a host-side decode and augmentation bottleneck.

```

1 from mlsysim.solvers import DataModel, TransformationModel
2 from mlsysim.hardware.registry import Hardware
3 from mlsysim.core.units import Q_
4
5 # 1. Define the ingestion demand (e.g., training a vision model)
6 # We need to feed 40,000 images per second to the GPU.
7 demand_rate = Q_("40000 1/s") * Q_("150 KB") # ~6 GB/s
8
9 # 2. Check if the A100 node's I/O path can handle the bandwidth
10 data_result = DataModel().solve(
11     workload_data_rate=demand_rate,
12     hardware=Hardware.Cloud.A100
13 )
14 print(f"I/O Stalled: {data_result.is_stalled}") # False
15
16 # 3. Check if 16 CPU workers can decode + augment fast enough
17 # (850 images/s per worker, 150 KB per image)
18 cpu_throughput = 16 * 850 * Q_("150 KB/s")
19 transform_result = TransformationModel().solve(
20     batch_size=2048,
21     sample_size_bytes=Q_("150 KB"),
22     cpu_throughput=cpu_throughput,
23     accelerator_step_time=Q_("48 ms"),
24 )
25 print(f"CPU Stalled: {transform_result.is_bottleneck}") # True
26 print(f"Accel. Utilization: "
27       f"{transform_result.accelerator_utilization:.0%}") # 32%

```

Listing 11: Data Pipeline Analysis (Tier 1). Evaluating the “Data Wall”. This example separates I/O bandwidth bottlenecks (storage to GPU) from CPU transformation bottlenecks (decoding and augmenting data), in the spirit of Case Study S2 but with an under-provisioned 16-worker pipeline.

B.4.4 Scenario D: SLA-Driven Hardware Synthesis (Tier 2)

Scenario: You are provisioning hardware for a new LLM application. You do not start with a GPU SKU; you start with the business requirement that the application should emit a token every 30 milliseconds. The `SynthesisSolver` inverts the same first-order Roofline model used for evaluation: given `Models.Language.Llama3_8B`, the target latency, and precision, it returns the memory-bandwidth and compute-throughput requirements that candidate hardware must clear.

```
1 from mlsysim.solvers import SynthesisSolver
2 from mlsysim.models.registry import Models
3 from mlsysim.core.units import Q_
4
5 # We want to serve Llama-3 8B.
6 # Our strict SLA dictates an inter-token latency of 30 ms.
7 # What is the minimum hardware capability required to achieve this?
8
9 solver = SynthesisSolver()
10 requirements = solver.solve(
11     model=Models.Language.Llama3_8B,
12     target_latency=Q_("30 ms"),
13     precision="fp16"
14 )
15
16 # Output the hardware requirements
17 print(f"Required HBM Bandwidth: {requirements.required_bw.to('GB/s'):.1f}")
18 # Required HBM Bandwidth: 535.3 GB / second
19 print(f"Required Compute: {requirements.required_flops.to('TFLOPs/s'):.1f}")
20 # Required Compute: 1.1 TFLOPs / second
```

Listing 12: **SLA-Driven Hardware Synthesis (Tier 2).** Instead of evaluating a known piece of hardware, the `SynthesisSolver` works backward. Given a strict latency SLA, it algebraically inverts the Roofline equations to output first-order minimum memory bandwidth and compute throughput requirements for procurement screening.

B.4.5 Scenario E: Speculative Decoding Speedup (Tier 1)

Scenario: You are serving a large frontier model and want to know how much faster inference would be if you used a smaller, cheaper draft model to speculatively decode tokens. The first call establishes the autoregressive baseline for the target model on a single B200, which holds the FP16 Llama-3 70B weights that an 80 GB H100 cannot. The second call adds a draft model and an acceptance-rate assumption, allowing `ServingModel` to account for draft generation and target-model verification before reporting the effective inter-token latency and speedup.

```
1 from mlsysim.solvers import ServingModel
2 from mlsysim.hardware.registry import Hardware
3 from mlsysim.models.registry import Models
4
5 # Serve Llama-3 70B using Llama-3 8B as a draft model
6 target_model = Models.Language.Llama3_70B
7 draft_model = Models.Language.Llama3_8B
8 hardware = Hardware.Cloud.B200
9
10 solver = ServingModel()
11
12 # Standard Autoregressive Decoding
13 base_result = solver.solve(target_model, hardware, seq_len=2048, batch_size=1)
14 print(f"Standard ITL: {base_result.itl.to('ms'):.2f}")
15 # Standard ITL: 18.53 millisecond
16
17 # Speculative Decoding
18 spec_result = solver.solve(
19     target_model, hardware, seq_len=2048, batch_size=1,
20     draft_model=draft_model,
21     draft_acceptance_rate=0.75
22 )
23 print(f"Speculative ITL: {spec_result.itl.to('ms'):.2f}")
```

```

24 # Speculative ITL: 8.75 millisecond
25 print(f"Speedup: {(base_result.itl / spec_result.itl).magnitude:.2f}x")
26 # Speedup: 2.12x

```

Listing 13: **Speculative Decoding Analysis (Tier 1)**. The `ServingModel` natively supports algorithmic optimizations like speculative decoding. By providing a draft model and an expected acceptance rate, the solver calculates the effective inter-token latency (ITL) by modeling the compute-bound draft phase and the memory-bound verification phase.

B.4.6 Scenario F: The Full-Stack Scorecard (Tier 2)

Scenario: Rather than invoking individual solvers, you want the single “will it run, is it fast enough, and what does it cost” verdict for a deployment. An executable `Scenario` is a named bundle of workload, target system, SLA, and operating context. Calling `evaluate()` computes the Feasibility, Performance, and Macro levels and returns a `SystemEvaluation` whose scorecard preserves the reason for each pass or failure; `validate()` instead raises at the first failing level.

```

1 import mlsysim
2
3 # A pre-built executable scenario: chatbot serving with an SLA + grid context
4 evaluation = mlsysim.Scenarios.ChatbotServing.evaluate()
5 print(evaluation.scorecard())
6 # (abridged)
7 # Level 1: Feasibility [PASS] Model fits in memory (16.1 GB / 85.9 GB)
8 # Level 2: Performance [PASS] Latency: 5.60 millisecond (Target: 500 ms)
9 # Level 3: Macro/Economics [PASS] Annual Carbon: 2946.3 kg | TCO: $10,407
10 print(evaluation.passed_all) # True

```

Listing 14: **Three-Level Scorecard (Tier 2)**. `Scenario.evaluate()` composes the resolver chain automatically and returns a `SystemEvaluation`: Level 1 checks memory feasibility, Level 2 reports latency against the SLA, and Level 3 reports cost and carbon.

B.4.7 Scenario G: Automated Parallelism Search (Tier 3)

Scenario: You have a fixed cluster and a large model and want the framework to *find* the best 3D-parallelism split rather than specifying it by hand. The Tier 3 `ParallelismOptimizer` enumerates power-of-two $TP \times PP \times DP$ factorizations of the available devices, computes per-GPU training state for each candidate, and discards layouts that exceed HBM. The result keeps the best MFU configuration and a ranked candidate list, making the search procedure visible rather than a black-box recommendation.

```

1 import mlsysim
2 from mlsysim.solvers import ParallelismOptimizer
3
4 fleet = mlsysim.Systems.Clusters.Research_256 # 256x H100
5 result = ParallelismOptimizer().solve(
6     mlsysim.Models.Language.Llama3_70B, fleet,
7     batch_size=2048, precision="fp16", efficiency=0.5,
8 )
9 best = result.best_config
10 print(f"TP={best['tp']}, PP={best['pp']}, DP={best['dp']}")
11 # TP=2, PP=16, DP=8
12 print(f"MFU: {result.best_mfu:.3f}") # MFU: 0.458
13 for c in result.top_candidates[1:3]:
14     k = c["config"]
15     print(f"TP={k['tp']}, PP={k['pp']}, DP={k['dp']} {k['mfu']:.3f}")
16 # TP=2, PP=32, DP=4 0.456
17 # TP=2, PP=128, DP=1 0.452

```

Listing 15: **Parallelism Optimizer (Tier 3)**. The optimizer enumerates valid $TP/PP/DP$ factorizations of the cluster, discards configurations whose per-GPU training state exceeds HBM, and returns the highest-MFU survivor with its ranked candidate list.

B.4.8 Scenario H: From Datacenter to Microcontroller (Tier 1)

Scenario: The same dimensionally strict engine that sizes an H100 fleet also sizes a microcontroller. The loop changes only the registry-backed model and hardware entries; `Engine.solve()` still lowers the workload demand, reads the supply constraints, and returns feasibility, bottleneck regime, latency, and per-inference energy. This is the benefit of the demand–supply contract: datacenter and TinyML examples share the same code path while drawing facts from very different parts of the registry.

```
1 from mlsysim.engine.engine import Engine
2 from mlsysim.hardware.registry import Hardware
3 from mlsysim.models.registry import Models
4
5 deployments = [
6     (Models.Vision.LeNet1, Hardware.Tiny.ESP32_S3),
7     (Models.Vision.MobileNetV2, Hardware.Edge.JetsonOrinNano),
8     (Models.Vision.MobileNetV2_Alpha0_5, Hardware.Edge.Coral),
9 ]
10 for model, hw in deployments:
11     perf = Engine.solve(model, hw, batch_size=1, precision="int8")
12     print(f"{hw.name:24s} {perf.bottleneck}-bound "
13           f"{perf.latency.to('ms'):.2f} {perf.energy.to('mJ'):.2f}")
14 # ESP32-S3 (AI) Compute-bound 1.03 millisecond 0.13 millijoule
15 # NVIDIA Jetson Orin Nano Memory-bound 0.80 millisecond 3.74 millijoule
16 # Google Coral Edge TPU Memory-bound 1.28 millisecond 0.77 millijoule
```

Listing 16: Edge and TinyML Deployment (Tier 1). The identical `Engine.solve` call evaluates milliwatt-class devices: a LeNet variant fits in the ESP32-S3’s on-chip SRAM, while MobileNet variants run on a Jetson Orin Nano and a Coral Edge TPU. Energy is per inference.